

12. 병렬 스트림

#0.강의/1.자바로드맵/7.자바-고급3편

- /단일 스트림
- /스레드 직접 사용
- /스레드 풀 사용
- /Fork/Join 패턴
- /Fork/Join 프레임워크1 - 소개
- /Fork/Join 프레임워크2 - 작업 훔치기
- /작업 훔치기 알고리즘
- /Fork/Join 프레임워크3 - 공용 풀
- /자바 병렬 스트림
- /병렬 스트림 사용시 주의점1
- /병렬 스트림 사용시 주의점2
- /정리

단일 스트림

자바 병렬 스트림을 제대로 이해하려면, 우리가 앞서 학습한 스트림은 물론이고, 멀티스레드, Fork/Join 프레임워크에 대한 기본 지식이 필요하다. 멀티스레드에 대한 부분은 **김영한의 실전 자바 고급 1편** 강의를 참고하자.

여기서는 단일 스트림부터 시작해서 멀티스레드, 스레드 풀, Fork/Join 프레임워크, 그리고 자바 병렬 스트림으로 이어지는 전체 과정을 예제를 통해 점진적으로 설명하겠다.

병렬 스트림 준비 예제

병렬 스트림을 학습하기 전에 먼저 로깅 유틸리티와 무거운 작업을 시뮬레이션하는 클래스를 준비하자.

```
package util;

import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;

public abstract class MyLogger {

    private static final DateTimeFormatter formatter =
        DateTimeFormatter.ofPattern("HH:mm:ss.SSS");
```

```

    public static void log(Object obj) {
        String time = LocalDateTime.now().format(formatter);
        System.out.printf("%s [%9s] %s\n", time,
Thread.currentThread().getName(), obj);
    }
}

```

- `MyLogger` 클래스는 현재 시간, 스레드 이름, 그리고 전달받은 객체를 로그로 출력한다. 이 클래스를 사용하면 어떤 스레드에서 어떤 작업이 실행되는지 시간과 함께 확인할 수 있다.

```

package parallel;

import util.MyLogger;

import java.util.concurrent.ForkJoinPool;

public class HeavyJob {

    public static int heavyTask(int i) {
        MyLogger.log("calculate " + i + " -> " + i * 10);
        try {
            Thread.sleep(1000);
        } catch (InterruptedException e) {
            throw new RuntimeException(e);
        }
        return i * 10;
    }

    public static int heavyTask(int i, String name) {
        MyLogger.log("[ " + name + " ] " + i + " -> " + i * 10);
        try {
            Thread.sleep(1000);
        } catch (InterruptedException e) {
            throw new RuntimeException(e);
        }
        return i * 10;
    }
}

```

- `HeavyJob` 클래스는 오래 걸리는 작업을 시뮬레이션하는데, 각 작업은 1초 정도 소요된다고 가정하겠다. 입력 값에 10을 곱한 결과를 반환하며, 작업이 실행될 때마다 로그를 출력한다.
- 추가로 로그를 찍어서 어느 스레드가 이 작업을 처리 중인지 확인할 수 있다.

예제1 - 단일 스트림

먼저 **단일 스트림**(sequential stream)으로 `IntStream.rangeClosed(1, 8)` 에서 나온 1부터 8까지의 숫자 각각에 대해 `heavyTask()` 를 순서대로 수행해보자.

```
package parallel;

import java.util.stream.IntStream;

import static util.MyLogger.log;

public class ParallelMain1 {

    public static void main(String[] args) {
        long startTime = System.currentTimeMillis();

        int sum = IntStream.rangeClosed(1, 8)
            .map(HeavyJob::heavyTask)
            .reduce(0, (a, b) -> a + b); // sum()

        long endTime = System.currentTimeMillis();
        log("time: " + (endTime - startTime) + "ms, sum: " + sum);
    }
}
```

- `map(HeavyJob::heavyTask)` 로 1초씩 걸리는 작업을 8번 순차로 호출하므로, 약 8초가 소요된다.
- 마지막에 `reduce(0, (a, b) -> a + b)`, 또는 `sum()` 으로 최종 결과를 합산한다.
- 결과적으로 **단일 스레드**(`main` 스레드)에서 작업을 순차적으로 수행하기 때문에 로그에도 `[main]` 스레드만 표시된다.

실행 결과

```
15:27:03.155 [    main] calculate 1 -> 10
15:27:04.162 [    main] calculate 2 -> 20
15:27:05.164 [    main] calculate 3 -> 30
15:27:06.170 [    main] calculate 4 -> 40
```

```
15:27:07.175 [    main] calculate 5 -> 50
15:27:08.181 [    main] calculate 6 -> 60
15:27:09.186 [    main] calculate 7 -> 70
15:27:10.192 [    main] calculate 8 -> 80
15:27:11.201 [    main] time: 8067ms, sum: 360
```

- 실제 출력 로그를 보면 `calculate 1 -> 10`, `calculate 2 -> 20`, ..., `calculate 8 -> 80` 등이 순서대로 찍힌다.
- 전체 시간이 8초 정도 걸리는 것을 확인할 수 있다.

8초는 너무 오래 걸린다. 스레드를 사용해서 실행 시간을 단축해보자.

스레드 직접 사용

앞서 하나의 메인 스레드로 `1 ~ 8`의 범위를 모두 계산했다.

이제 여러 스레드를 동시에 사용해서 작업을 더 빨리 처리해보자.

각 스레드는 한 번에 하나의 작업만 처리할 수 있다. 따라서 `1 ~ 8`을 처리하는 큰 단위의 작업을 더 작은 단위의 작업으로 분할해야 한다. 여기서는 `1 ~ 8`의 큰 작업을 `1 ~ 4`, `5 ~ 8`과 같이 절반으로 분할해서 두 개의 스레드로 처리해보자.

예제2 - 스레드 직접 사용

```
package parallel;

import static util.MyLogger.log;

public class ParallelMain2 {

    public static void main(String[] args) throws InterruptedException {
        long startTime = System.currentTimeMillis();

        // 1. Fork 작업을 분할한다.
        SumTask task1 = new SumTask(1, 4);
        SumTask task2 = new SumTask(5, 8);
        Thread thread1 = new Thread(task1, "thread-1");
        Thread thread2 = new Thread(task2, "thread-2");
```

```

// 2. 분할한 작업을 처리한다.
thread1.start();
thread2.start();

// 3. join - 처리한 결과를 합친다.
thread1.join();
thread2.join();
log("main 스레드 대기 완료");

int sum = task1.result + task2.result;
long endTime = System.currentTimeMillis();
log("time: " + (endTime - startTime) + "ms, sum: " + sum);
}

static class SumTask implements Runnable {
    int startValue;
    int endValue;
    int result = 0;

    public SumTask(int startValue, int endValue) {
        this.startValue = startValue;
        this.endValue = endValue;
    }

    @Override
    public void run() {
        log("작업 시작");
        int sum = 0;
        for (int i = startValue; i <= endValue; i++) {
            int calculated = HeavyJob.heavyTask(i);
            sum += calculated;
        }
        result = sum;

        log("작업 완료 result=" + result);
    }
}
}

```

- SumTask 는 Runnable 을 구현했고, 내부에서 1초씩 걸리는 heavyTask() 를 루프 돌면서 합산한다.
- new SumTask(1, 4), new SumTask(5, 8) 을 통해 작업을 두 개로 분할한다.
- thread1.start(), thread2.start() 로 각 스레드가 동시에 작업을 시작하고, thread1.join(),

`thread2.join()` 으로 두 스레드가 끝날 때까지 `main` 스레드가 대기한다.

- 작업 완료 후, `task1`, `task2` 의 결과를 더해서 최종 합계(`sum`)를 구한다.

실행 결과

```
15:29:27.018 [ thread-1] 작업 시작
15:29:27.018 [ thread-2] 작업 시작
15:29:27.024 [ thread-1] calculate 1 -> 10
15:29:27.024 [ thread-2] calculate 5 -> 50
15:29:28.025 [ thread-2] calculate 6 -> 60
15:29:28.029 [ thread-1] calculate 2 -> 20
15:29:29.030 [ thread-2] calculate 7 -> 70
15:29:29.032 [ thread-1] calculate 3 -> 30
15:29:30.036 [ thread-2] calculate 8 -> 80
15:29:30.036 [ thread-1] calculate 4 -> 40
15:29:31.043 [ thread-1] 작업 완료 result=100
15:29:31.043 [ thread-2] 작업 완료 result=260
15:29:31.043 [      main] main 스레드 대기 완료
15:29:31.047 [      main] time: 4042ms, sum: 360
```

- `thread-1`, `thread-2` 가 작업을 분할해서 처리했기 때문에 8초의 작업을 4초로 줄일 수 있었다.
- 하지만 이렇게 스레드를 직접 사용하면 스레드 수가 늘어나면 코드가 복잡해지고, 예외 처리, 스레드 풀 관리 등 추가 관리 포인트가 생기는 문제가 있다.

스레드 풀 사용

이번에는 자바가 제공하는 `ExecutorService` 를 사용해서 더 편리하게 병렬 처리를 해보자.

참고: 스레드 풀 관련해서는 김영한의 실전 자바 고급 1편 - 멀티스레드와 동시성 강의의 스레드 풀과 Executor 프레임워크 1, 2를 참고하자.

예제3 - 스레드 풀

```
package parallel;
```

```

import java.util.concurrent.*;

import static util.MyLogger.log;

public class ParallelMain3 {

    public static void main(String[] args) throws InterruptedException,
    ExecutionException {
        // 스레드 풀을 준비한다.
        ExecutorService es = Executors.newFixedThreadPool(2);

        long startTime = System.currentTimeMillis();

        // 1. 작업을 분할한다.
        SumTask task1 = new SumTask(1, 4);
        SumTask task2 = new SumTask(5, 8);

        // 2. 분할한 작업을 처리한다.
        Future<Integer> future1 = es.submit(task1);
        Future<Integer> future2 = es.submit(task2);

        // 3. join - 처리한 결과를 합친다. get: 결과가 나올 때 까지 대기한다.
        Integer result1 = future1.get();
        Integer result2 = future2.get();

        int sum = result1 + result2;
        long endTime = System.currentTimeMillis();
        log("time: " + (endTime - startTime) + "ms, sum: " + sum);

        es.close();
    }

    static class SumTask implements Callable<Integer> {
        int startValue;
        int endValue;

        public SumTask(int startValue, int endValue) {
            this.startValue = startValue;
            this.endValue = endValue;
        }

        @Override
        public Integer call() {

```

```

        log("작업 시작");
        int sum = 0;
        for (int i = startValue; i <= endValue; i++) {
            int calculated = HeavyJob.heavyTask(i);
            sum += calculated;
        }
        log("작업 완료 result=" + sum);
        return sum;
    }
}
}

```

- `Executors.newFixedThreadPool(2)` 로 스레드 풀을 만든다. 이 스레드 풀은 최대 2개의 스레드를 제공한다.
- `new SumTask(1, 4)`, `new SumTask(5, 8)` 을 통해 작업을 두 개로 분할한다.
- `submit(Callable)` 로 스레드 풀에 작업을 맡기면 `Future` 객체를 반환 받는다.
- 메인 스레드는 `future.get()` 을 통해 실제 계산 결과가 반환될 때까지 대기(`join` 과 유사)한다.

이 예제는 스레드 풀과 `Future` 을 사용해서 결과값을 반환받는 방식으로 구현되었다. 작업이 완료되면 `Future` 의 `get()` 메서드를 통해 결과를 얻는다. 참고로 `get()` 메서드는 블로킹 메서드이다. 이전 예제와 마찬가지로 2개의 스레드가 병렬로 계산을 처리하므로 약 4초가 소요된다.

실행 결과

```

15:31:19.238 [pool-1-thread-1] 작업 시작
15:31:19.238 [pool-1-thread-2] 작업 시작
15:31:19.244 [pool-1-thread-1] calculate 1 -> 10
15:31:19.244 [pool-1-thread-2] calculate 5 -> 50
15:31:20.247 [pool-1-thread-1] calculate 2 -> 20
15:31:20.247 [pool-1-thread-2] calculate 6 -> 60
15:31:21.248 [pool-1-thread-2] calculate 7 -> 70
15:31:21.253 [pool-1-thread-1] calculate 3 -> 30
15:31:22.252 [pool-1-thread-2] calculate 8 -> 80
15:31:22.258 [pool-1-thread-1] calculate 4 -> 40
15:31:23.258 [pool-1-thread-2] 작업 완료 result=260
15:31:23.258 [pool-1-thread-1] 작업 완료 result=100
15:31:23.270 [      main] time: 4040ms, sum: 360

```

- 이전 예제처럼 스레드가 2개이므로 각각 4개씩 나눠 처리한다.
- `Future` 로 반환값을 쉽게 받아올 수 있기 때문에, 결과값을 합산하는 과정이 더 편리해졌다.
- 하지만 여전히 코드 레벨에서 분할/병합 로직을 직접 짜야 하고, 스레드 풀 생성과 관리도 개발자가 직접해야 한

다.

Fork/Join 패턴

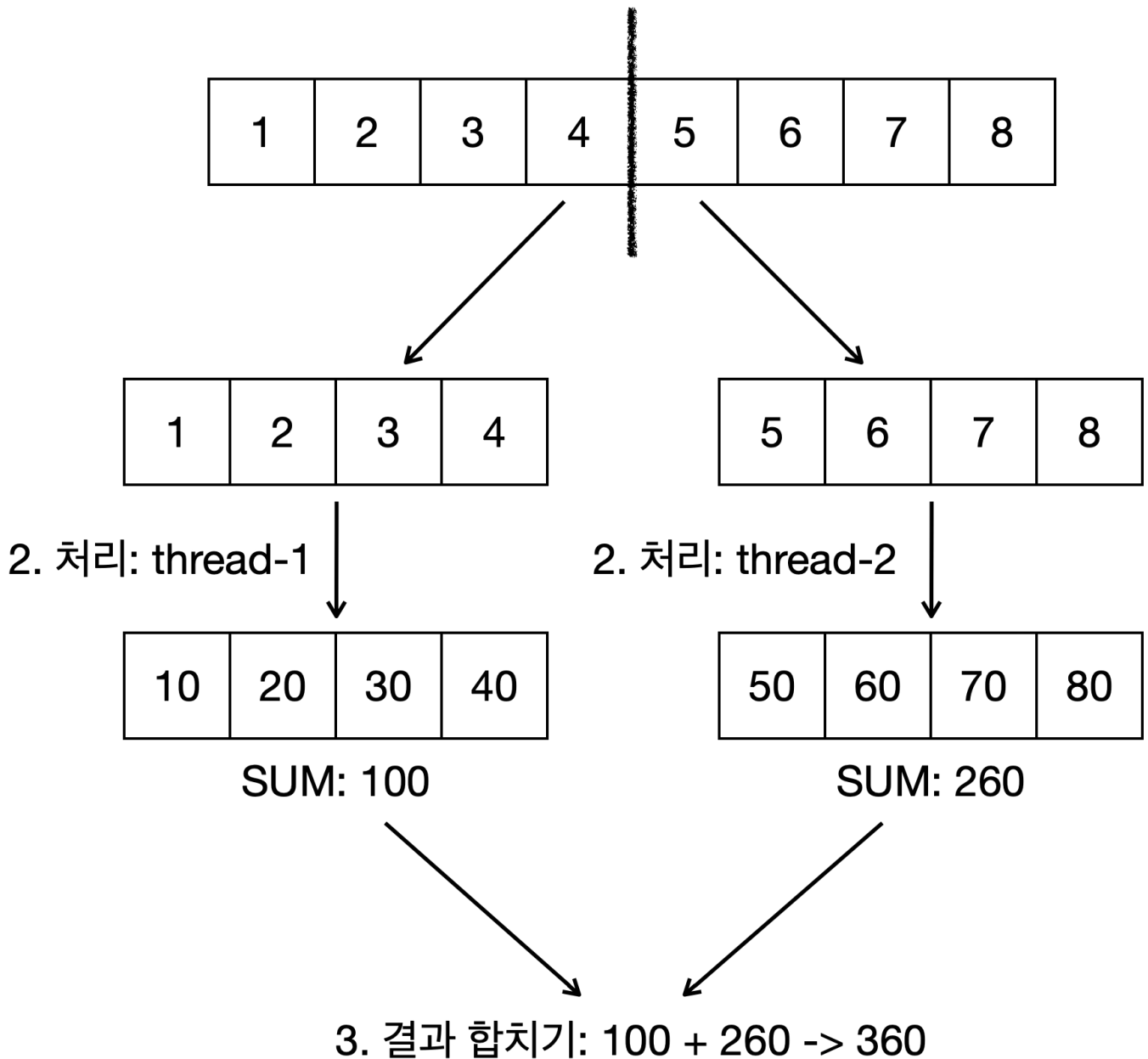
분할(Fork), 처리(Execute), 모음(Join)

스레드는 한 번에 하나의 작업을 처리할 수 있다. 따라서 하나의 큰 작업을 여러 스레드가 처리할 수 있는 작은 단위의 작업으로 분할(Fork)해야 한다. 그리고 이렇게 분할한 작업을 각각의 스레드가 처리(Execute)하는 것이다. 각 스레드의 분할된 작업 처리가 끝나면 분할된 결과를 하나로 모아야(Join) 한다.

이렇게 **분할(Fork) → 처리(Execute) → 모음(Join)**의 단계로 이루어진 멀티스레딩 패턴을 **Fork/Join 패턴**이라고 부른다. 이 패턴은 병렬 프로그래밍에서 매우 효율적인 방식으로, 복잡한 작업을 병렬적으로 처리할 수 있게 해준다.

지금까지 우리는 이런 과정을 다음과 같이 직접 처리했다. 우리가 진행했던 예제를 그림과 함께 다시 정리해보자.

1. 작업 분할



1. 작업 분할(Fork)

1 ~ 8 분할

- 1 ~ 4: thread-1 처리
- 5 ~ 8: thread-2 처리

1 ~ 8의 작업을 절반으로 분할하자. 그래서 1 ~ 4의 작업은 thread-1이 처리하고, 5 ~ 8의 작업은 thread-2가 처리하는 것이다. 이렇게 하면 작업의 수를 늘려서 여러 스레드가 동시에 많은 작업을 처리할 수 있다. 예제에서는 하나의 스레드가 처리하던 작업을 두 개의 스레드가 처리하므로 처리 속도를 최대 2배로 늘릴 수 있다.

이렇게 큰 작업을 여러 작은 작업으로 쪼개어(Fork) 각각의 스레드나 작업 단위로 할당하는 것을 포크(Fork)라 한다.

참고로 포크라는 이름은 식당에서 사용하는 포크가 여러 갈래로 나뉘어 있는 모양을 떠올려보면 된다. 이처럼 하나의 큰

작업을 여러 작은 작업으로 분할하는 것을 포크라 한다.

2. 처리(Execute)

- 1 ~ 4 처리(thread-1)
 - 1 → 10
 - 2 → 20
 - 3 → 30
 - 4 → 40
 - 결과: $10 + 20 + 30 + 40 = 100$
- 5 ~ 8 처리(thread-2)
 - 5 → 50
 - 6 → 60
 - 7 → 70
 - 8 → 80
 - 결과: $50 + 60 + 70 + 80 = 260$

thread-1, thread-2 는 분할된 각각의 작업을 처리한다.

3. Join 모음, 결과 합치기

분할된 작업들이 모두 끝나면, 각 스레드 혹은 작업 단위별 결과를 하나로 합쳐야한다.

예제에서는 `thread1.join()`, `thread2.join()` 을 통해 모든 스레드가 종료되길 기다린 뒤, `task1.result` + `task2.result` 로 최종 결과를 계산한다.

Join은 이렇게 갈라진 작업들이 모두 끝난 뒤, 다시 합류하여 하나로 결과를 모으는 모습을 의미한다.

정리

지금까지 작업을 직접 분할하고, 처리하고, 처리된 결과를 합쳤다.

이러한 분할 → 처리(작업 병렬 실행) → 모음의 과정을 더 편리하게 구현할 수 있는 방법은 없을까?

자바는 **Fork/Join 프레임워크**를 제공해서 개발자가 이러한 패턴을 더 쉽게 구현할 수 있도록 지원한다.

Fork/Join 프레임워크1 - 소개

Fork/Join 프레임워크 소개

자바의 Fork/Join 프레임워크는 자바 7부터 도입된 `java.util.concurrent` 패키지의 일부로, 멀티코어 프로세

서를 효율적으로 활용하기 위한 병렬 처리 프레임워크이다. 주요 개념은 다음과 같다.

분할 정복(Divide and Conquer) 전략

- 큰 작업(task)을 작은 단위로 재귀적으로 분할(fork)
- 각 작은 작업의 결과를 합쳐(join) 최종 결과를 생성
- 멀티코어 환경에서 작업을 효율적으로 분산 처리

작업 훔치기(Work Stealing) 알고리즘

- 각 스레드는 자신의 작업 큐를 가짐
- 작업이 없는 스레드는 다른 바쁜 스레드의 큐에서 작업을 "훔쳐와서" 대신 처리
- 부하 균형을 자동으로 조절하여 효율성 향상

주요 클래스

Fork/Join 프레임워크를 이루는 주요 클래스는 다음과 같다.

- `ForkJoinPool`
- `ForkJoinTask`
 - `RecursiveTask`
 - `RecursiveAction`

ForkJoinPool

- Fork/Join 작업을 실행하는 특수한 `ExecutorService` 스레드 풀
- 작업 스케줄링 및 스레드 관리를 담당
- 기본적으로 사용 가능한 프로세서 수 만큼 스레드 생성
 - 예) CPU 코어가 10 코어면 10개의 스레드 생성
- 쉽게 이야기해서 **분할 정복과 작업 훔치기에 특화된 스레드 풀**이다.

```
// 기본 풀 생성 (프로세서 수에 맞춰 스레드 생성)
ForkJoinPool pool = new ForkJoinPool();

// 특정 병렬 수준으로 풀 생성
ForkJoinPool customPool = new ForkJoinPool(4);
```

ForkJoinTask

- `ForkJoinTask` 는 Fork/Join 작업의 기본 추상 클래스다.
- `Future` 를 구현했다.

- 개발자는 주로 다음 두 하위 클래스를 구현해서 사용한다.
 - `RecursiveTask<V>`: 결과를 반환하는 작업
 - `RecursiveAction`: 결과를 반환하지 않는 작업(`void`)

RecursiveTask / RecursiveAction의 구현 방법

- `compute()` 메서드를 재정의해서 필요한 작업 로직을 작성한다.
- 일반적으로 일정 기준(임계값)을 두고, **작업 범위가 작으면 직접 처리하고, 크면 작업을 둘로 분할하여 각각 병렬로 처리하도록 구현한다.**

fork() / join() 메서드

- `fork()`: 현재 스레드에서 다른 스레드로 작업을 **분할**하여 보내는 동작(비동기 실행)
- `join()`: 분할된 작업이 끝날 때까지 기다린 후 결과를 가져오는 동작

참고: Fork/Join 프레임워크를 실무에서 직접적으로 다루는 일은 드물다. 따라서 이렇게 있다 정도만 알아두고 넘어가자.

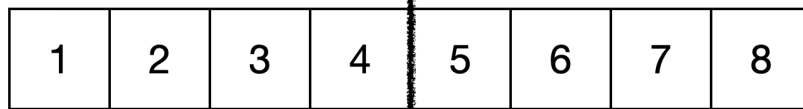
개념 정도만 대략 알아두면 충분하다.

Fork/Join 프레임워크 활용

실제 Fork/Join 프레임워크를 사용해서 우리가 앞서 처리한 예시를 개발해보자.

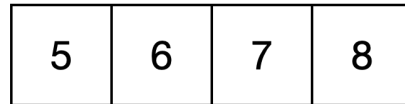
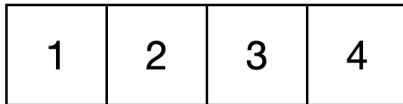
기본적인 처리 방식은 다음과 같다.

1. [분할]: 작업이 큼

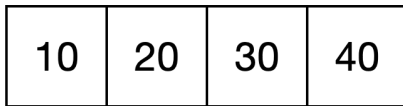


LEFT

RIGHT

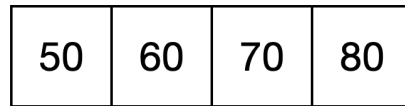


2. [처리]: 작업이 작음



SUM: 100

2. [처리]: 작업이 작음



SUM: 260

3. 결과 합치기: 100 + 260 -> 360

핵심은 작업의 크기가 임계값 보다 크면 분할하고, 임계값 보다 같거나 작으면 직접 처리하는 것이다.

예를 들어 작업의 크기가 8이고, 임계값이 4라고 가정해보자.

1. **Fork**: 작업의 크기가 8이면 임계값을 넘었다. 따라서 작업을 절반으로 분할한다.
2. **Execute**: 다음으로 작업의 크기가 4라면 임계값의 범위 안에 들어오므로 작업을 분할하지 않고, 처리한다.
3. **Join**: 최종 결과를 합친다.

Fork/Join 프레임워크를 사용하려면 `RecursiveTask.compute()` 메서드를 재정의해야 한다.

다음에 작성한 `SumTask` 는 `RecursiveTask<Integer>` 를 상속받아 리스트의 합을 계산하는 작업을 병렬로 처리하는 클래스이다. 이 클래스는 Fork/Join 프레임워크의 **분할 정복** 전략을 구현한다.

```
package parallel.forkjoin;

import parallel.HeavyJob;

import java.util.List;
import java.util.concurrent.RecursiveTask;

import static util.MyLogger.log;
```

```

public class SumTask extends RecursiveTask<Integer> {
    private static final int THRESHOLD = 4; // 임계값

    private final List<Integer> list;

    public SumTask(List<Integer> list) {
        this.list = list;
    }

    @Override
    protected Integer compute() {
        // 작업 범위가 작으면 직접 합산
        if (list.size() <= THRESHOLD) {
            log("[처리 시작] " + list);
            int sum = list.stream()
                .mapToInt(HeavyJob::heavyTask)
                .sum();
            log("[처리 완료] " + list + " -> sum: " + sum);
            return sum;
        } else {
            // 작업 범위가 크면 반으로 나누어 병렬 처리
            int mid = list.size() / 2;
            List<Integer> leftList = list.subList(0, mid);
            List<Integer> rightList = list.subList(mid, list.size());
            log("[분할] " + list + " -> LEFT" + leftList + ", RIGHT" +
rightList);

            SumTask leftTask = new SumTask(leftList);
            SumTask rightTask = new SumTask(rightList);

            // 왼쪽 작업은 다른 스레드에서 처리
            leftTask.fork();
            // 오른쪽 작업은 현재 스레드에서 처리
            int rightResult = rightTask.compute();

            // 왼쪽 작업 결과를 기다림
            int leftResult = leftTask.join();
            // 왼쪽과 오른쪽 작업 결과를 합침
            int joinSum = leftResult + rightResult;
            log("LEFT[" + leftResult + "] + RIGHT[" + rightResult + "] -> sum:
" + joinSum);
            return joinSum;
        }
    }
}

```

```

    }
}
}

```

- **THRESHOLD (임계값)**: 작업을 더 이상 분할하지 않고 직접 처리할 리스트의 크기를 정의한다. 여기서는 4로 설정되어, 리스트 크기가 4 이하일 때 직접 계산한다. 4보다 크면 작업을 분할한다.
- **작업 분할**: 리스트의 크기가 임계값보다 크면, 리스트를 반으로 나누어 `leftList`와 `rightList`로 분할한다.
- **fork(), compute()**
 - `fork()`는 왼쪽 작업을 다른 스레드에 위임하여 병렬로 처리한다.
 - `compute()`는 오른쪽 작업을 현재 스레드에서 직접 수행한다(재귀 호출).
- **join()**: 분할된 왼쪽 작업이 완료될 때까지 기다린 후 결과를 가져온다.
- **결과 합산**: 왼쪽과 오른쪽 결과를 합쳐 최종 결과를 반환한다.

이렇게 구현한 코드를 실제 실행해보자.

```

package parallel.forkjoin;

import java.util.List;
import java.util.concurrent.ExecutionException;
import java.util.concurrent.ForkJoinPool;
import java.util.stream.IntStream;

import static util.MyLogger.log;

public class ForkJoinMain1 {

    public static void main(String[] args) throws ExecutionException,
        InterruptedException {
        List<Integer> data = IntStream.rangeClosed(1, 8)
            .boxed()
            .toList();

        log("[생성] " + data);

        // ForkJoinPool 생성 및 작업 수행
        ForkJoinPool pool = new ForkJoinPool(10);

        long startTime = System.currentTimeMillis();
        SumTask task = new SumTask(data);

        // 병렬로 합을 구한 후 결과 출력
    }
}

```



```

        int result = pool.invoke(task);
        pool.close();
        long endTime = System.currentTimeMillis();
        log("time: " + (endTime - startTime) + "ms, sum: " + result);
        log("pool: " + pool);
    }
}

```

1. **데이터 생성:** `IntStream.rangeClosed(1, 8)` 를 사용해 1부터 8까지의 숫자 리스트를 생성한다.
2. **ForkJoinPool 생성:**
 - `new ForkJoinPool(10)` 으로 최대 10개의 스레드를 사용할 수 있는 풀을 생성한다.
 - 참고로 기본 생성자(`new ForkJoinPool()`)를 사용하면 시스템의 프로세서 수에 맞춰 스레드가 생성된다.
3. **invoke():** 메인 스레드가 `pool.invoke(task)` 를 호출하면 `SumTask` 를 스레드 풀에 전달한다. `SumTask` 는 `ForkJoinPool` 에 있는 별도의 스레드에서 실행된다. 메인 스레드는 작업이 완료될 때까지 기다린 후 결과를 받는다.
4. **pool.close():** 더 이상 작업이 없으므로 풀을 종료한다.
5. **결과 출력:** 계산된 리스트의 합과 실행 시간을 출력한다.

실행 결과

```

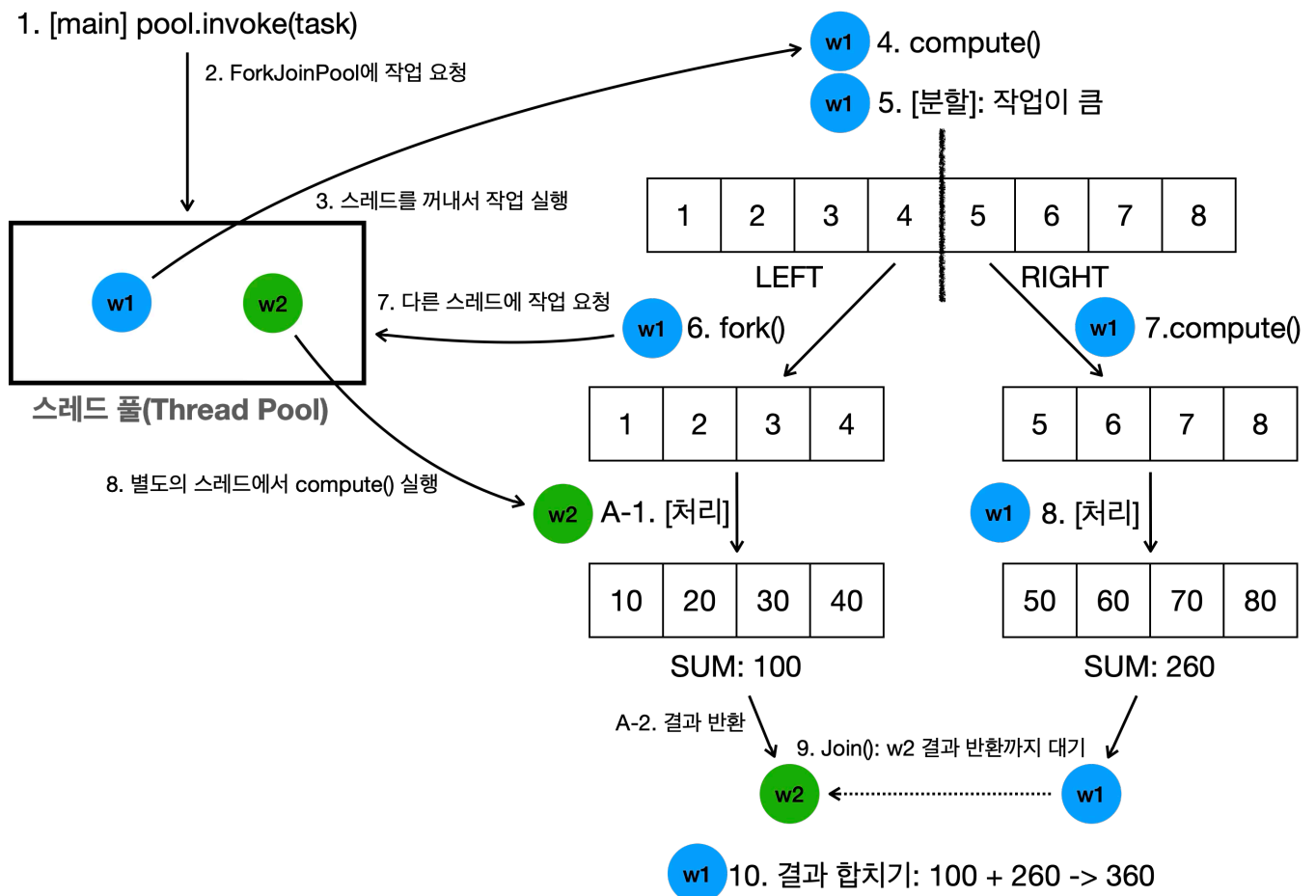
15:15:15.919 [      main] [생성] [1, 2, 3, 4, 5, 6, 7, 8]
15:15:15.928 [ForkJoinPool-1-worker-1] [분할] [1, 2, 3, 4, 5, 6, 7, 8] ->
LEFT[1, 2, 3, 4], RIGHT[5, 6, 7, 8]
15:15:15.929 [ForkJoinPool-1-worker-1] [처리 시작] [5, 6, 7, 8]
15:15:15.929 [ForkJoinPool-1-worker-2] [처리 시작] [1, 2, 3, 4]
15:15:15.932 [ForkJoinPool-1-worker-1] calculate 5 -> 50
15:15:15.932 [ForkJoinPool-1-worker-2] calculate 1 -> 10
15:15:16.936 [ForkJoinPool-1-worker-1] calculate 6 -> 60
15:15:16.936 [ForkJoinPool-1-worker-2] calculate 2 -> 20
15:15:17.941 [ForkJoinPool-1-worker-1] calculate 7 -> 70
15:15:17.941 [ForkJoinPool-1-worker-2] calculate 3 -> 30
15:15:18.946 [ForkJoinPool-1-worker-1] calculate 8 -> 80
15:15:18.946 [ForkJoinPool-1-worker-2] calculate 4 -> 40
15:15:19.955 [ForkJoinPool-1-worker-1] [처리 완료] [5, 6, 7, 8] -> sum: 260
15:15:19.955 [ForkJoinPool-1-worker-2] [처리 완료] [1, 2, 3, 4] -> sum: 100
15:15:19.960 [ForkJoinPool-1-worker-1] LEFT[100] + RIGHT[260] -> sum: 360
15:24:02.152 [      main] time: 4040ms, sum: 360
15:24:02.153 [      main] pool:

```

```
java.util.concurrent.ForkJoinPool@57baedf[Terminated, parallelism = 10, size = 0, active = 0, running = 0, steals = 2, tasks = 0, submissions = 0]
```

작업이 2개로 분할 되어서 총 4초의 시간이 걸린 것을 확인할 수 있다.

실행 그림



작업 시작

1. main 스레드가 invoke(task) 를 호출해서
2. ForkJoinPool 에 작업을 요청
3. 스레드 풀은 스레드를 꺼내서 작업을 실행 여기서는 ForkJoinPool-1-worker-1 스레드가 실행됨, 줄여서 w1 이라고 표현하겠음
4. w1 스레드는 task(SumTask) 의 compute() 를 호출함

작업 분할

[w1] [분할] [1, 2, 3, 4, 5, 6, 7, 8] -> LEFT[1, 2, 3, 4], RIGHT[5, 6, 7, 8]

5. 리스트 크기가 THRESHOLD(4) 보다 크므로 분할됨.

- `[1, 2, 3, 4, 5, 6, 7, 8]` 이 `LEFT[1, 2, 3, 4]` 와 `RIGHT[5, 6, 7, 8]` 로 나뉨.
- 6. `w1` 은 분할할 왼쪽 리스트인 `LEFT[1, 2, 3, 4]` 는 `fork(leftTask)` 를 호출해서 다른 스레드가 작업을 처리하도록 요청함
- 7. `w1` 은 분할할 오른쪽 리스트인 `RIGHT[5, 6, 7, 8]` 는 자기 자신의 메서드인 `compute(rightTask)` 를 호출해서 자기 자신이 스스로 처리함 (재귀 호출)

병렬 처리

- 각 스레드가 동시에 `HeavyJob.heavyTask()`를 실행하며 병렬로 계산.
 - 8. `w1` 스레드가 `[5, 6, 7, 8]` 을 순서대로 처리, SUM: 260 (리스트 크기가 `THRESHOLD(4)` 이하)
 - A-1. `w2` 스레드가 `[1, 2, 3, 4]` 를 순서대로 처리, SUM: 100 (리스트 크기가 `THRESHOLD(4)` 이하)
- `[1, 2, 3, 4]` 작업의 합은 100, `[5, 6, 7, 8]` 의 작업의 합은 260

작업 완료:

- 9. 최종 결과의 합을 구하기 위해 `w1` 스레드는 `w2` 스레드의 작업에 `join()` 메서드를 호출해서 `w2` 의 결과를 기다림
- 10. 두 결과가 합쳐져 최종 합계 360이 계산됨

정리

- Fork/Join 프레임워크를 사용하면 `RecursiveTask` 를 통해 작업을 재귀적으로 분할하는 것을 확인할 수 있다. 여기서는 작업을 단순히 2개로만 분할해서 스레드도 동시에 2개만 사용할 수 있었다.
- `THRESHOLD` (임계값)을 더 줄여서 작업을 더 잘게 분할하면 더 많은 스레드를 활용할 수 있다. 물론 이 경우 풀의 스레드 수도 2개보다 더 많아야 효과가 있다.

참고 - 작업 훔치기

- 이번 그림은 단순하게 설명하게 위해 작업 훔치기는 생략했다.
- 실제로는 Fork/Join 풀의 스레드는 각자 자신의 작업 큐를 가진다. 그리고 자신의 작업이 없는 경우 다른 스레드에 대기중인 작업을 훔쳐서 대신 처리할 수 있다. 이 부분은 다음에 설명한다.

Fork/Join 프레임워크2 - 작업 훔치기

더 분할하기

이번에는 임계값을 줄여서 작업을 더 잘게 분할해보자. 다음 코드를 참고해서 `THRESHOLD` 값 4를 2로 변경하자. 그러면 8개의 작업이 4개의 작업으로 분할될 것이다.

```

public class SumTask extends RecursiveTask<Integer> {
    //private static final int THRESHOLD = 4; // 임계값
    private static final int THRESHOLD = 2; // 임계값 변경
    ...
}

```

그리고 ForkJoinMain1을 실행해보자.

실행 결과

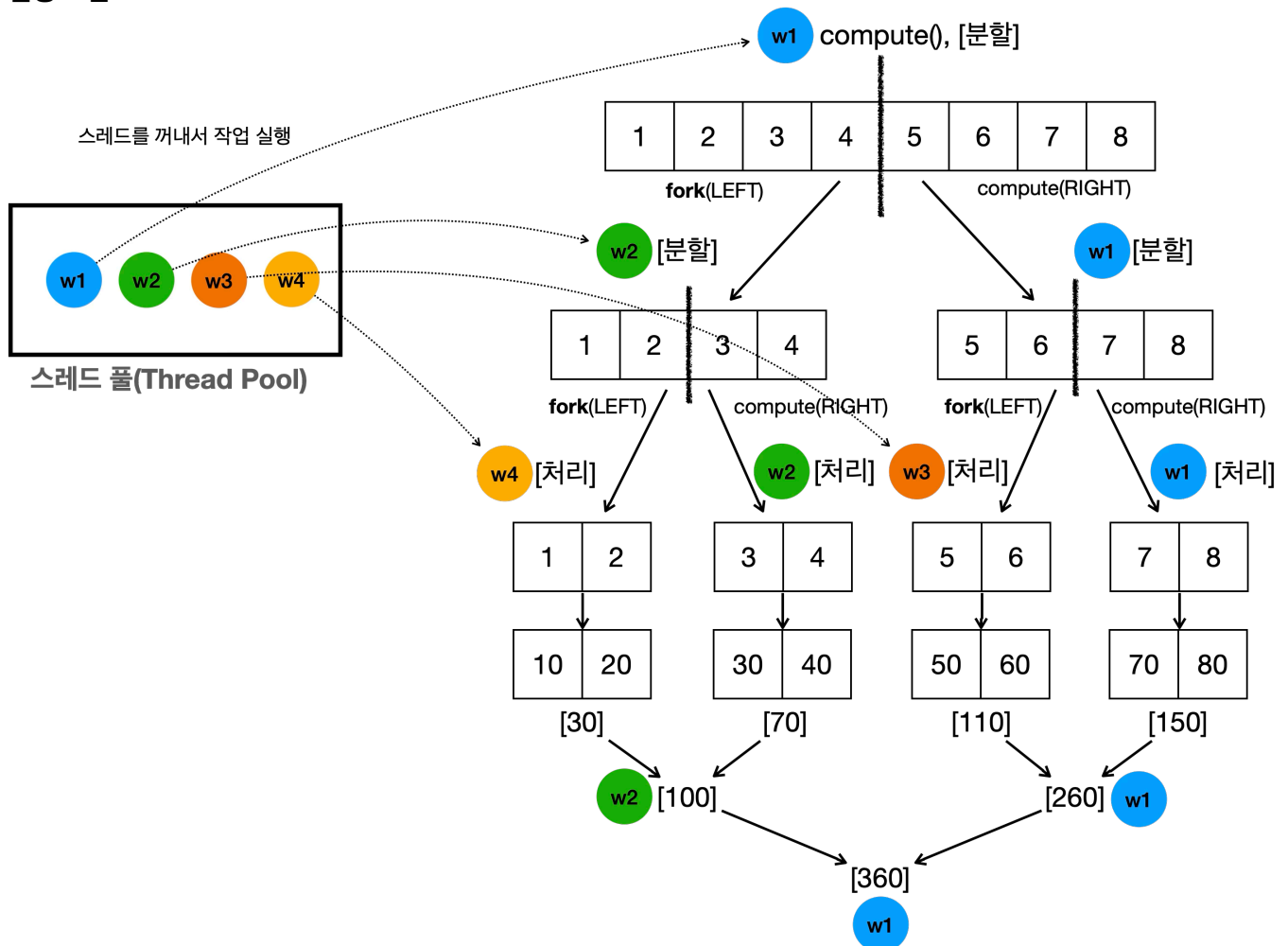
```

16:02:19.416 [      main] [생성] [1, 2, 3, 4, 5, 6, 7, 8]
16:02:19.425 [ForkJoinPool-1-worker-1] [분할] [1, 2, 3, 4, 5, 6, 7, 8] ->
LEFT[1, 2, 3, 4], RIGHT[5, 6, 7, 8]
16:02:19.425 [ForkJoinPool-1-worker-1] [분할] [5, 6, 7, 8] -> LEFT[5, 6],
RIGHT[7, 8]
16:02:19.425 [ForkJoinPool-1-worker-2] [분할] [1, 2, 3, 4] -> LEFT[1, 2],
RIGHT[3, 4]
16:02:19.425 [ForkJoinPool-1-worker-1] [처리 시작] [7, 8]
16:02:19.425 [ForkJoinPool-1-worker-2] [처리 시작] [3, 4]
16:02:19.425 [ForkJoinPool-1-worker-3] [처리 시작] [5, 6]
16:02:19.425 [ForkJoinPool-1-worker-4] [처리 시작] [1, 2]
16:02:19.430 [ForkJoinPool-1-worker-4] calculate 1 -> 10
16:02:19.430 [ForkJoinPool-1-worker-2] calculate 3 -> 30
16:02:19.430 [ForkJoinPool-1-worker-3] calculate 5 -> 50
16:02:19.430 [ForkJoinPool-1-worker-1] calculate 7 -> 70
16:02:20.435 [ForkJoinPool-1-worker-1] calculate 8 -> 80
16:02:20.435 [ForkJoinPool-1-worker-4] calculate 2 -> 20
16:02:20.435 [ForkJoinPool-1-worker-3] calculate 6 -> 60
16:02:20.435 [ForkJoinPool-1-worker-2] calculate 4 -> 40
16:02:21.447 [ForkJoinPool-1-worker-3] [처리 완료] [5, 6] -> sum: 110
16:02:21.447 [ForkJoinPool-1-worker-1] [처리 완료] [7, 8] -> sum: 150
16:02:21.448 [ForkJoinPool-1-worker-4] [처리 완료] [1, 2] -> sum: 30
16:02:21.448 [ForkJoinPool-1-worker-2] [처리 완료] [3, 4] -> sum: 70
16:02:21.454 [ForkJoinPool-1-worker-2] LEFT[30] + RIGHT[70] -> sum: 100
16:02:21.454 [ForkJoinPool-1-worker-1] LEFT[110] + RIGHT[150] -> sum: 260
16:02:21.455 [ForkJoinPool-1-worker-1] LEFT[100] + RIGHT[260] -> sum: 360
16:02:21.455 [      main] time: 2030ms, sum: 360
16:02:21.456 [      main] pool:
java.util.concurrent.ForkJoinPool@57baeedf[Terminated, parallelism = 10, size
= 0, active = 0, running = 0, steals = 4, tasks = 0, submissions = 0]

```

임계값(THRESHOLD)을 4에서 2로 낮춘 결과, 작업이 더 잘게 분할되어 더 많은 스레드가 병렬로 작업을 처리하는 것을 확인할 수 있다. 여기서는 총 4개의 작업으로 분할되고, 2초의 시간이 소요되었다.

실행 그림



초기 분할

- 전체 배열 [1, 2, 3, 4, 5, 6, 7, 8] 이 먼저 [1, 2, 3, 4] 와 [5, 6, 7, 8] 두 부분으로 분할된다.
- 결과적으로 w1 스레드가 오른쪽 부분 [5, 6, 7, 8] 을 담당하고, w2 스레드가 왼쪽 부분 [1, 2, 3, 4] 를 담당한다.
 - w1 은 `fork(LEFT[1, 2, 3, 4])` 를 호출해서 왼쪽 부분을 w2 에 맡긴다.
 - w1 은 `compute(RIGHT[5, 6, 7, 8])` 을 호출해서 오른쪽 리스트를 스스로 담당한다. (재귀 호출)

추가 분할

- 임계값이 2로 설정되었으므로, 크기가 4인 두 부분은 각각 다시 분할된다.
- [5, 6, 7, 8] 은 [5, 6] 과 [7, 8] 로 분할된다.
 - w1 은 `fork(LEFT[5, 6])` 을 호출해서 왼쪽 부분을 w3 에 맡긴다.
 - w1 은 `compute(RIGHT[7, 8])` 를 호출해서 오른쪽 리스트를 스스로 담당한다. (재귀 호출)
- [1, 2, 3, 4] 는 [1, 2] 와 [3, 4] 로 분할된다.

- w2는 `fork(LEFT[1,2])`를 호출해서 왼쪽 부분을 w4에 맡긴다.
- w2은 `compute(RIGHT[3,4])`를 호출해서 오른쪽 리스트를 스스로 담당한다.

병렬 처리

- 각 작업 단위는 이제 임계값보다 작거나 같으므로 더 이상 분할되지 않고 처리된다.
- 임계값이 2 이하인 4개의 작업을 4개의 스레드가 동시에 처리한다.
 - w1: [7, 8] 처리
 - w2: [3, 4] 처리
 - w3: [5, 6] 처리
 - w4: [1, 2] 처리

계산 결과

- [1,2] -> [10,20] -> 합계[30]
- [3,4] -> [30,40] -> 합계[70]
- [5,6] -> [50,60] -> 합계[110]
- [7,8] -> [70,80] -> 합계[150]

결과 결합

- w1은 w3의 계산 결과를 기다린다. 그리고 110과 150의 결과를 결합하여 260을 얻고 반환한다.
- w2는 w4의 계산 결과를 기다린다. 그리고 30과 70의 결과를 결합하여 100을 얻고 반환한다.
- 마지막으로 w1은 w2의 계산 결과를 기다린다. 그리고 100과 260을 결합하여 최종 결과 360을 얻고 반환한다.

효율성 향상

- 임계값을 낮춤으로써 더 많은 스레드(총 4개)가 병렬로 작업을 처리했다.
- 이전 실행(임계값 4)에서는 2개의 스레드만 사용되었다.
- 로그를 보면 계산이 거의 동시에 시작되어 거의 동시에 완료된 것을 확인할 수 있다.

작업 훔치기 알고리즘

참고: 이 부분은 크게 중요하지 않으므로 참고로만 알아두자.

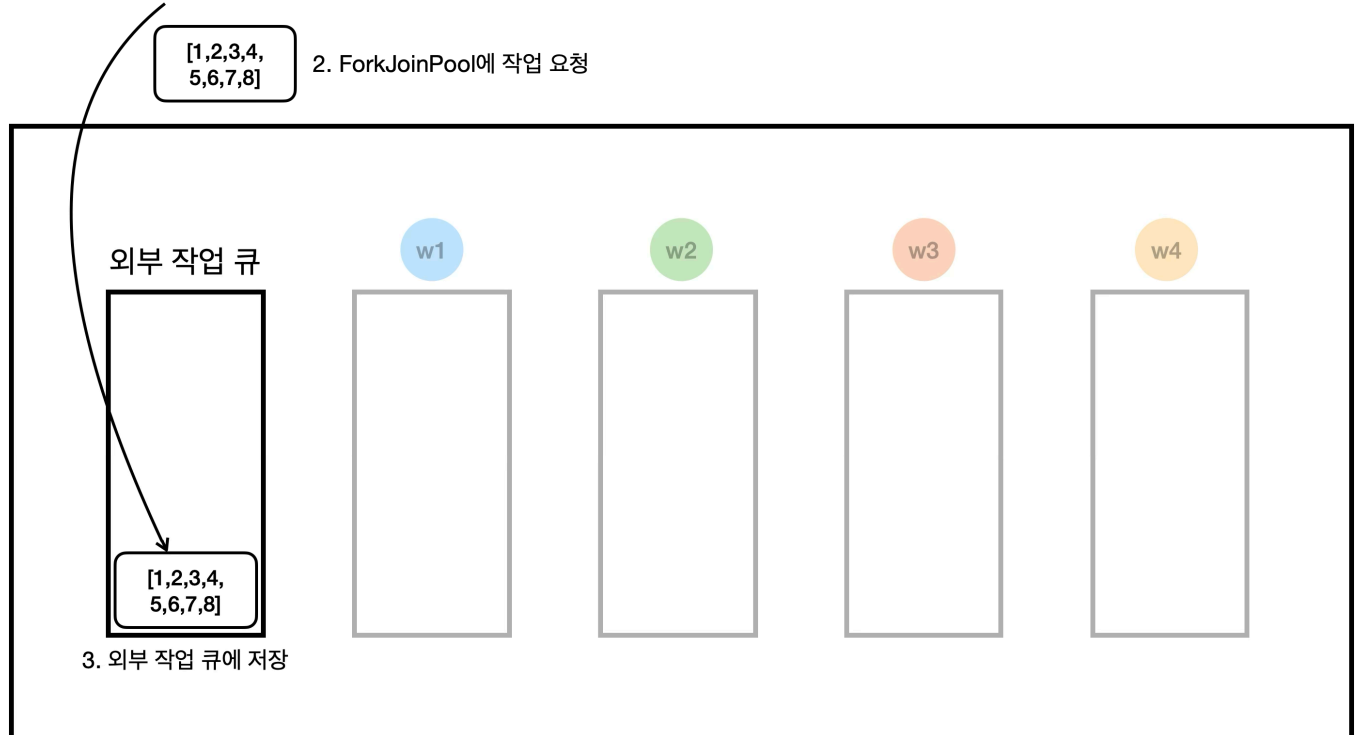
지금까지 설명을 단순화 하기 위해 작업 훔치기(Work-Stealing) 알고리즘은 설명하지 않았다. 이번에는 작업 훔치기에 대해 자세히 알아보자.

- Fork/Join 풀의 스레드는 각자 자신의 작업 큐를 가진다.

- 덕분에 작업을 큐에서 가져가기 위한 스레드간 경합이 줄어든다.
- 그리고 자신의 작업이 없는 경우, 그래서 스레드가 할 일이 없는 경우에 다른 스레드의 작업 큐에 대기중인 작업을 훔쳐서 대신 처리한다.

이번 예제의 작업 훔치기에 대해서 그림으로 자세히 알아보자.

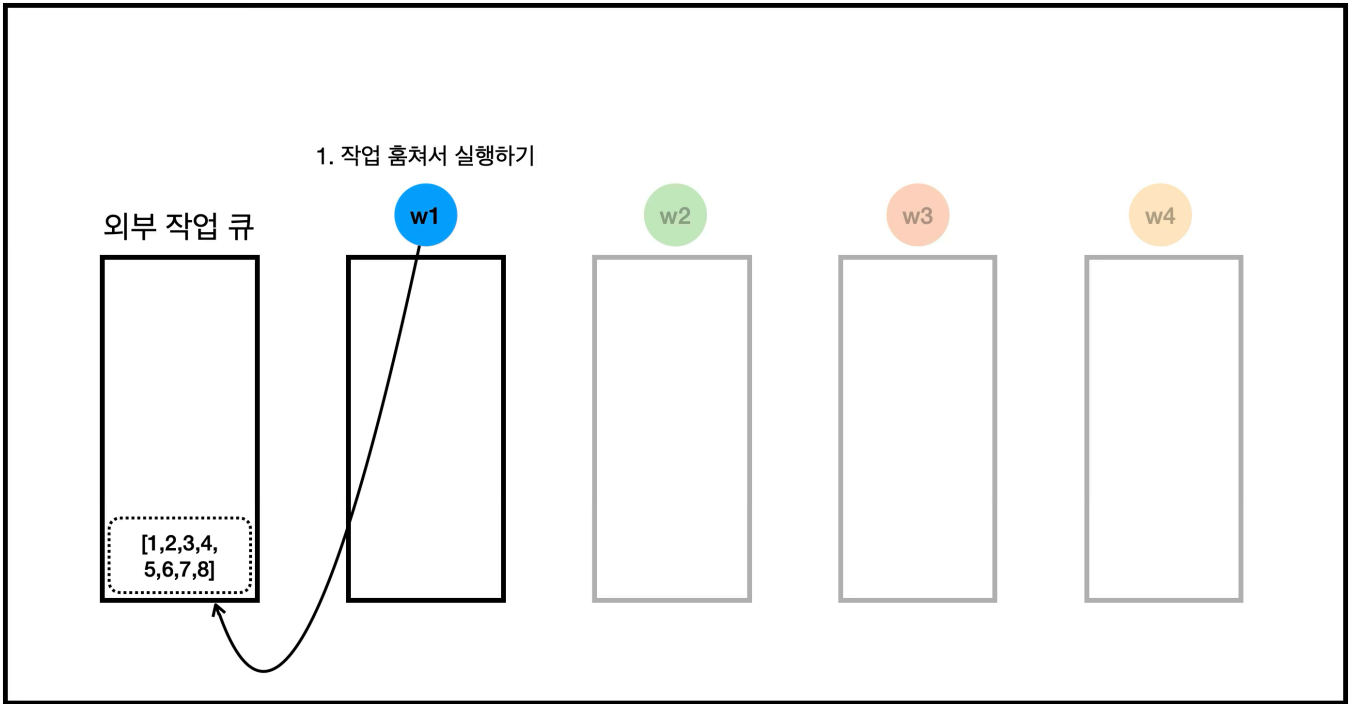
1. [main] pool.invoke(task)



포크 조인 풀(ForkJoinPool)

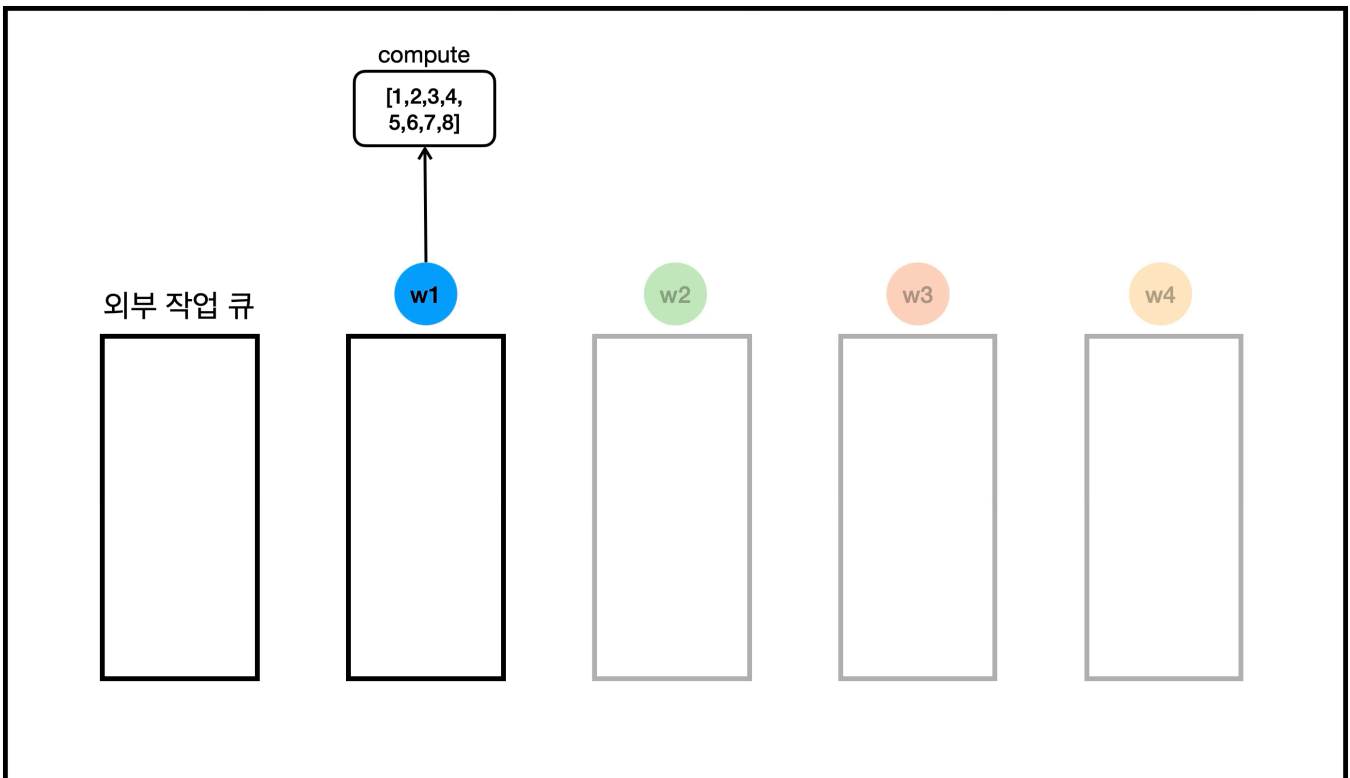
```
16:02:19.416 [      main] [생성] [1, 2, 3, 4, 5, 6, 7, 8]
```

- ForkJoinPool에 작업을 요청하면 ForkJoinPool 내부에 있는 외부 작업 큐에 작업이 저장된다.



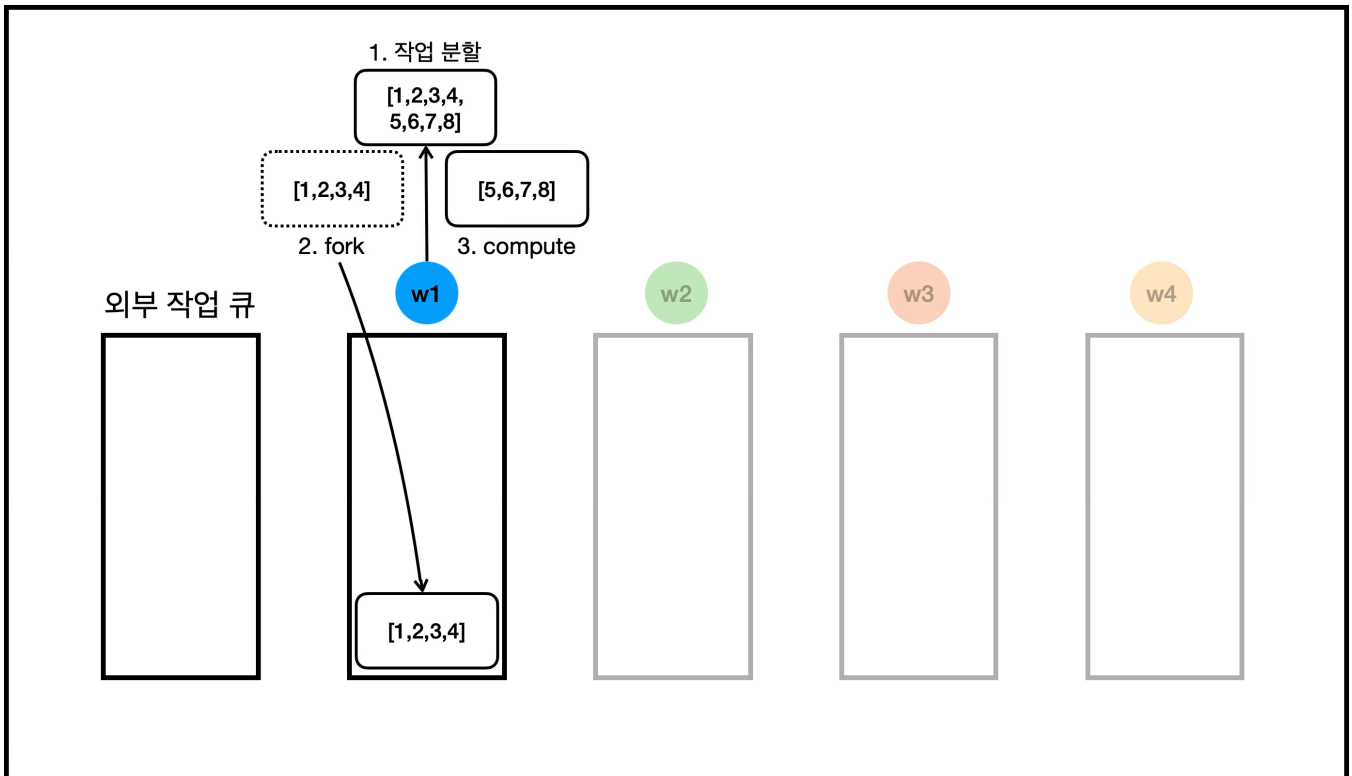
포크 조인 풀(ForkJoinPool)

- 포크 조인 풀의 스레드는 각자 자신의 작업 큐를 가진다.
- 포크 조인 풀의 스레드는 만약 자신이 할 일이 없고, 자신의 작업 큐에도 작업이 없는 경우 다른 작업 큐에 있는 작업을 훔쳐서 대신 처리할 수 있다.
- w1 스레드는 자신이 처리할 일이 없으므로 다른 작업 큐의 작업을 훔친다. 여기서는 외부 작업 큐에 들어 있는 작업을 훔쳐서 대신 처리한다.



포크 조인 풀(ForkJoinPool)

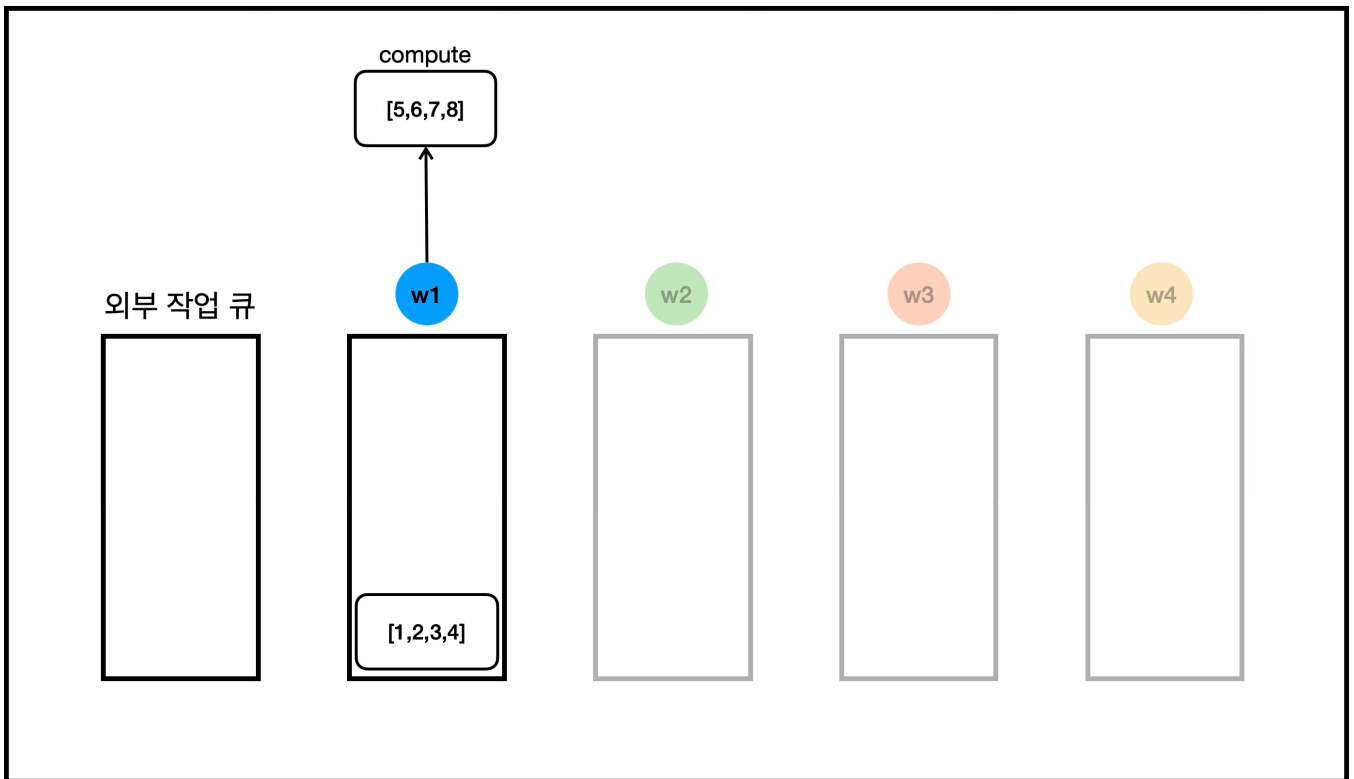
- w1 은 훔친 작업의 `compute()` 를 호출하면서 작업을 시작한다.



포크 조인 풀(ForkJoinPool)

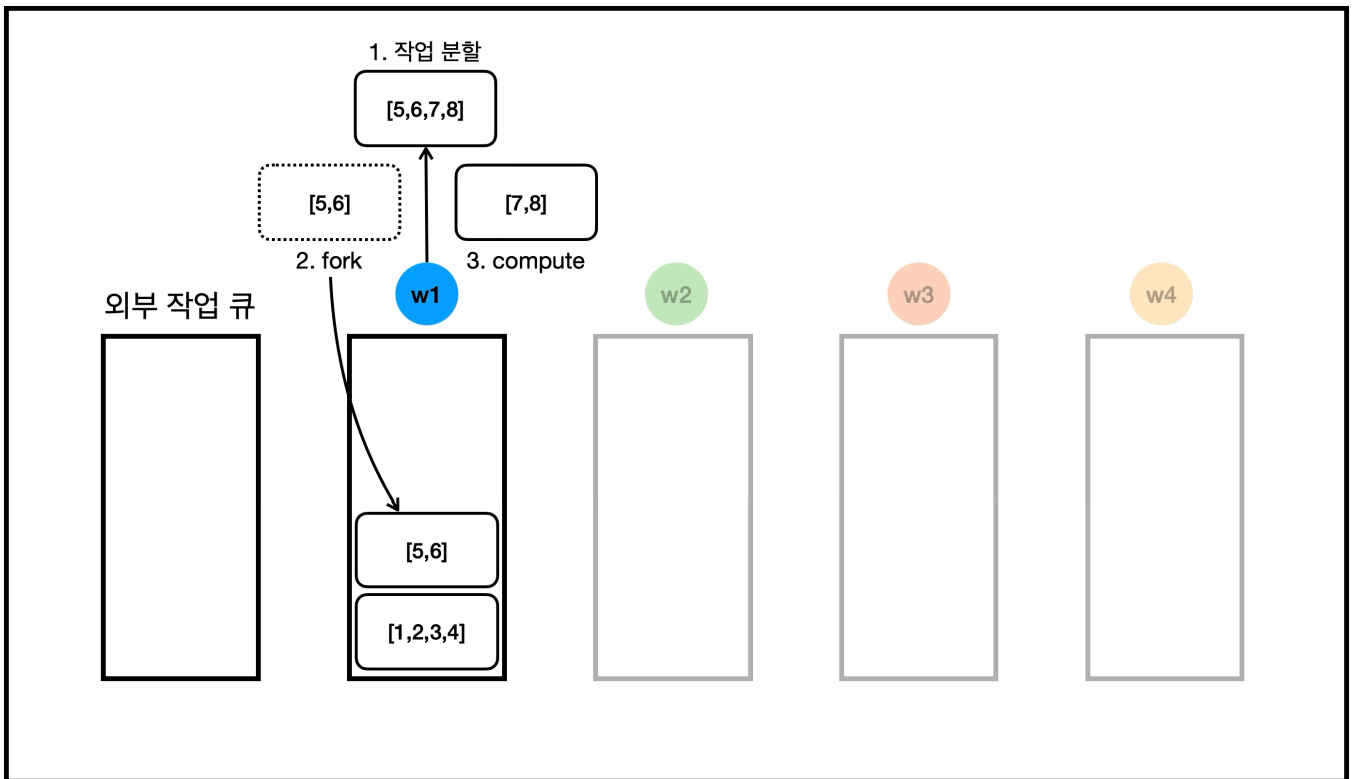
```
16:02:19.425 [ForkJoinPool-1-worker-1] [분할] [1, 2, 3, 4, 5, 6, 7, 8] ->
LEFT[1, 2, 3, 4], RIGHT[5, 6, 7, 8]
```

- w1 은 작업의 크기가 크다고 평가하고 작업을 둘로 분할한다.
- [1, 2, 3, 4] 의 작업은 fork를 호출해서 비동기로 다른 스레드가 실행해주길 기대한다.
- [5, 6, 7, 8] 의 작업은 compute를 호출해서 스스로 처리한다. (재귀 호출)
- 사실 `fork()` 는 스레드 자신의 작업 큐에 작업을 넣어두는 것이다.
 - 이후에 자신이 여유가 되면 스스로 보관한 작업을 처리하고, 자신이 여유가 없고 쉬는 스레드가 있다면 쉬는 스레드가 작업을 훔쳐가서 대신 처리한다.



포크 조인 풀(ForkJoinPool)

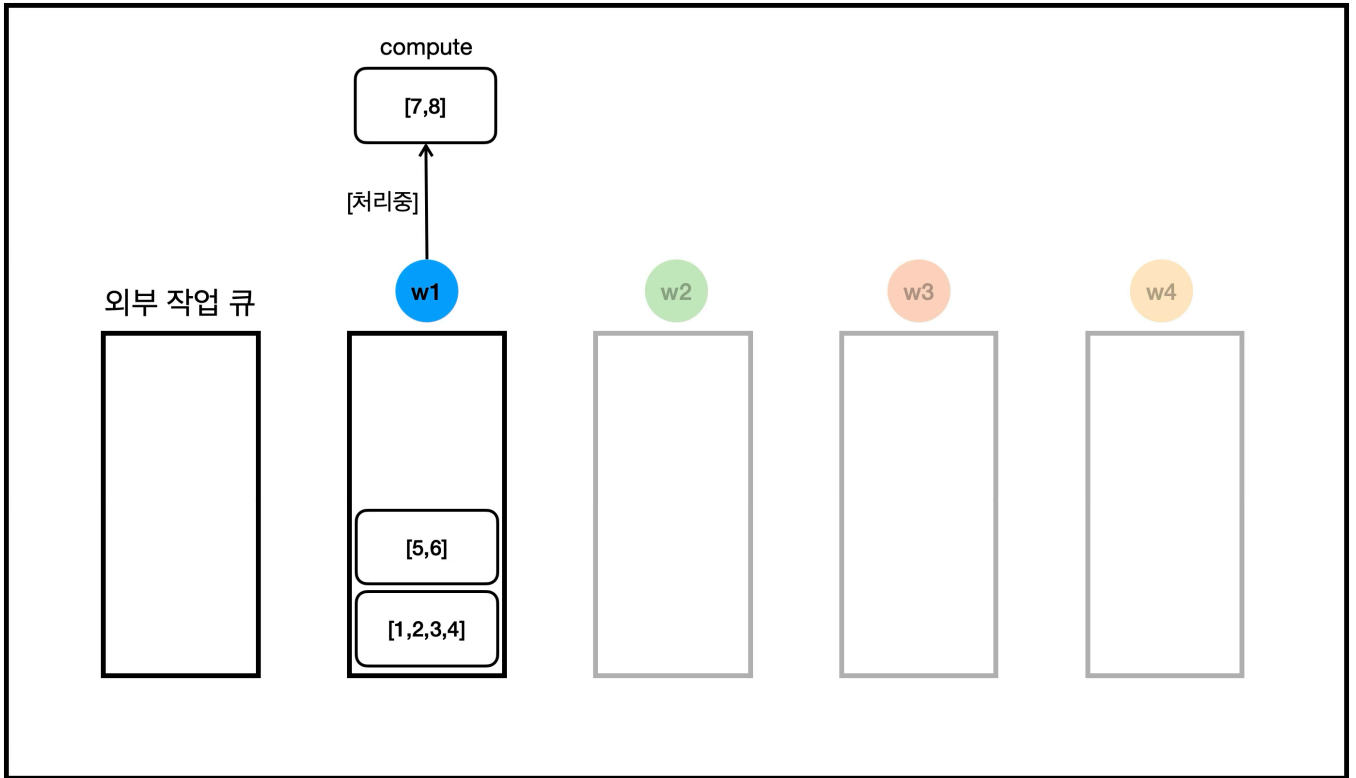
- w1 은 `compute([5, 6, 7, 8])` 를 호출했으므로 스스로 `[5, 6, 7, 8]` 을 처리해야 한다.
- w1 의 작업 큐에 있는 `[1, 2, 3, 4]` 작업은 아직 다른 스레드에서 훔쳐가지 않았다.
 - 이번 시나리오에서는 아직 훔쳐가지 않았지만, 실행 상황에 따라 이 시점에 작업을 훔쳐갈 수도 있다



포크 조인 풀(ForkJoinPool)

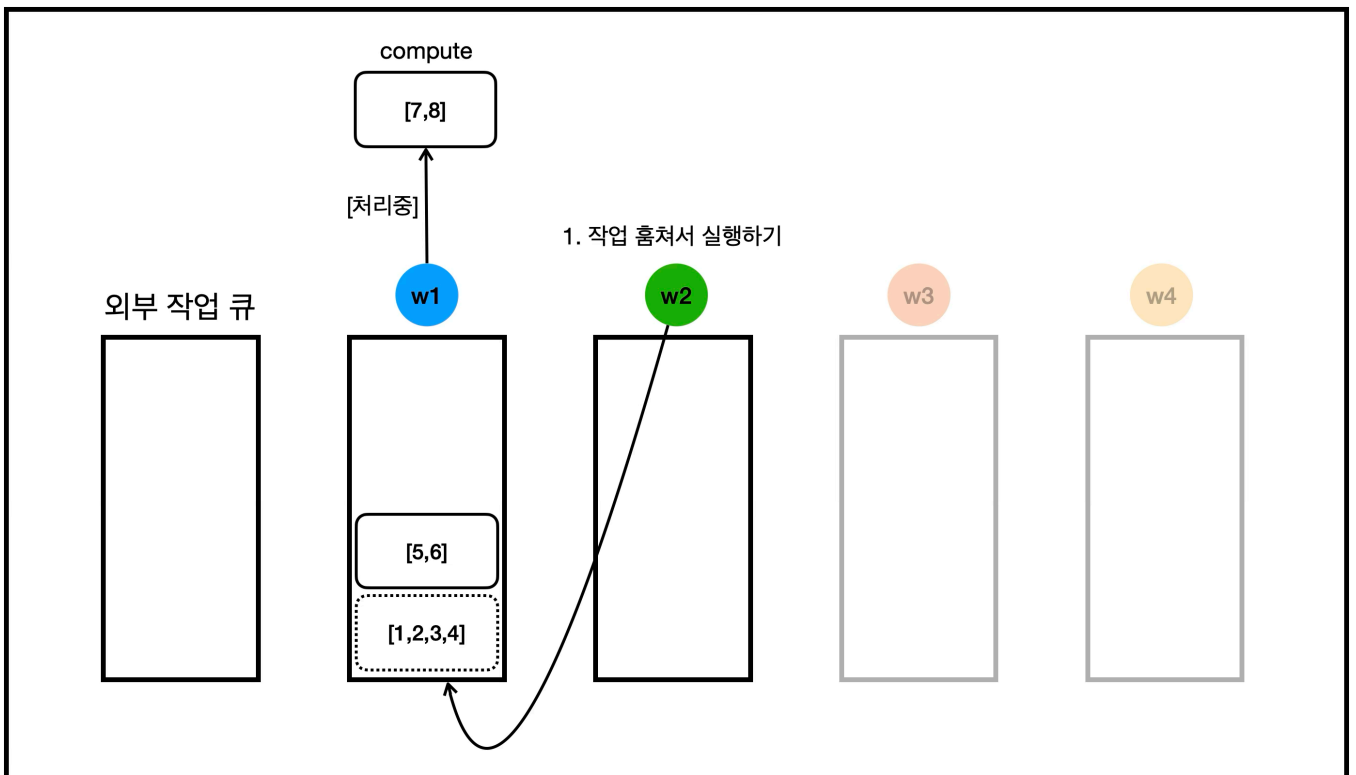
16:02:19.425 [ForkJoinPool-1-worker-1] [분할] [5, 6, 7, 8] -> LEFT[5, 6], RIGHT[7, 8]

- w1 은 [5, 6, 7, 8] 작업을 분할한다.
 - [5, 6] 은 Fork를 통해서 자신의 작업 큐에 보관한다.
 - [7, 8] 은 compute를 호출해서 스스로 처리한다.



포크 조인 풀(ForkJoinPool)

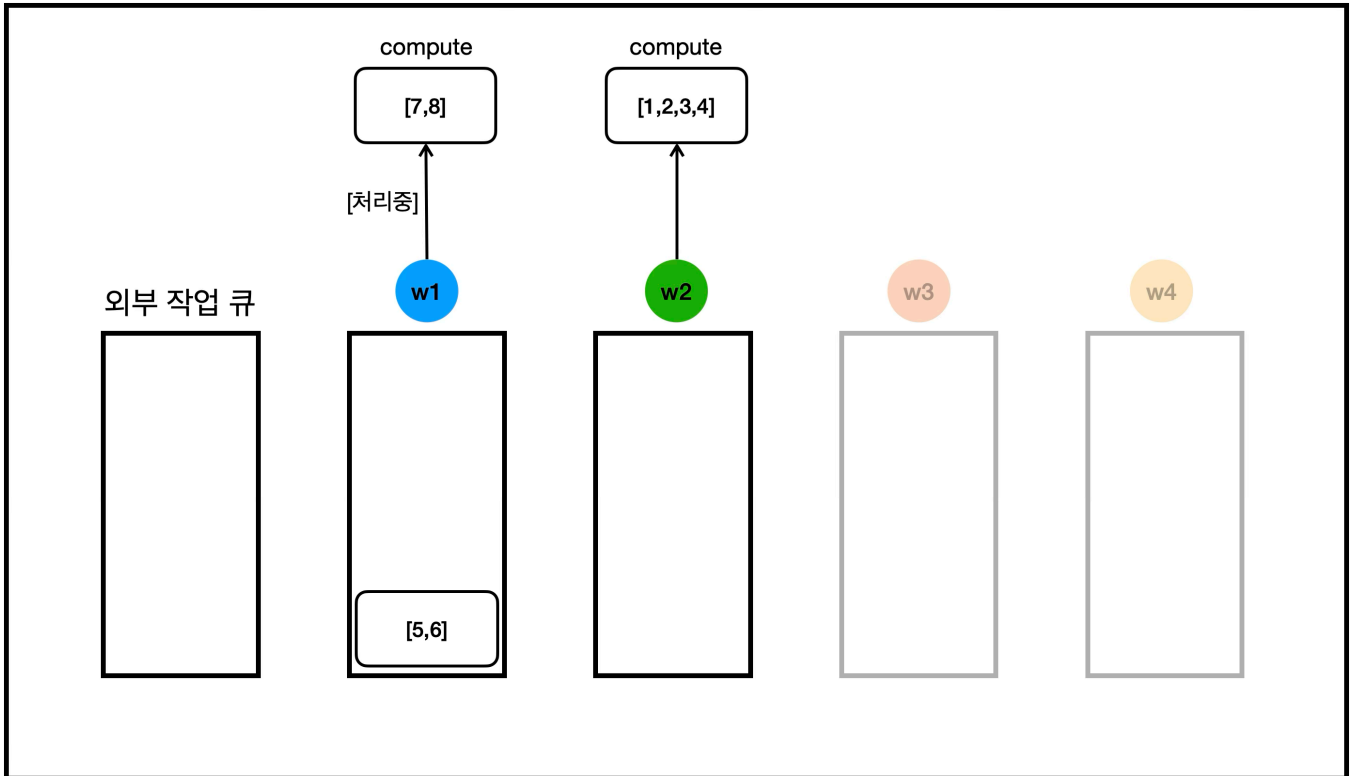
- w1 은 [7, 8] 의 처리를 시작한다.



포크 조인 풀(ForkJoinPool)

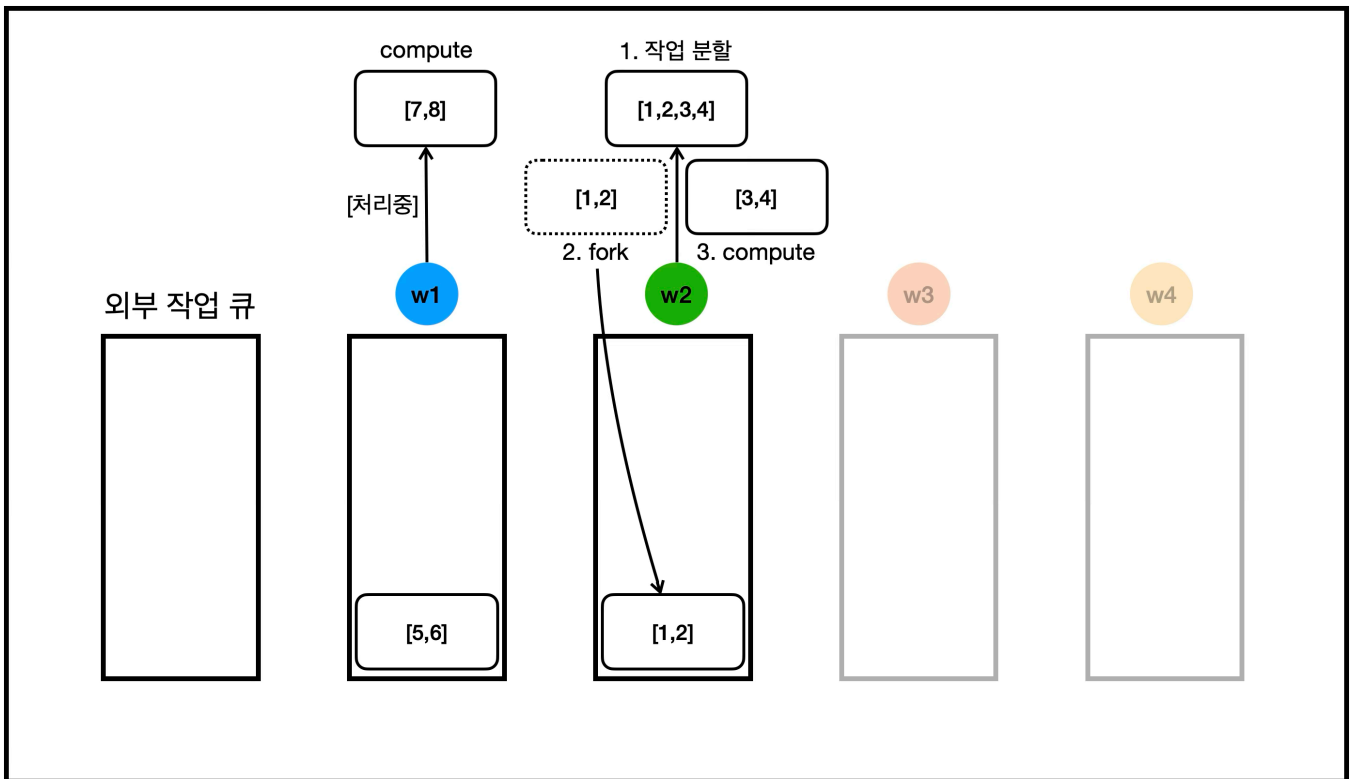
- w1 의 작업 큐에 작업이 2개나 대기 중이다. 쉬고 있는 w2 가 w1 의 작업 [1, 2, 3, 4] 를 훔친다.

- 참고로 여기에 있는 큐는 데크에 가깝다. 따라서 양쪽으로 넣고 뺄 수 있는 구조이다.
- 스레드 스스로 작업을 작업 큐에 보관하거나 꺼낼 때는 위에서, 다른 곳에서 훔칠 때는 아래 방향에서 훔친다.
 - 이런 구조 덕분에 경합이 덜 발생한다.



포크 조인 풀(ForkJoinPool)

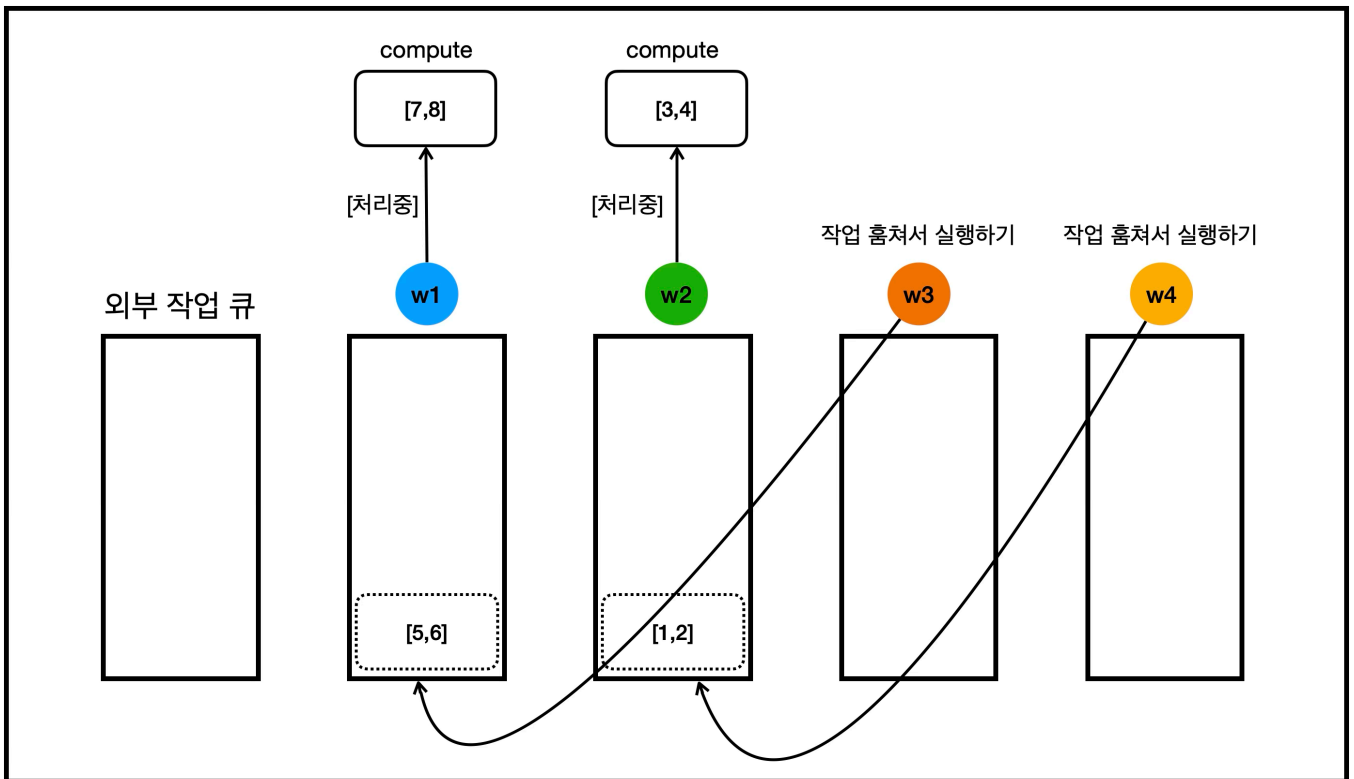
- w2 가 `compute()` 를 호출해서 `[1, 2, 3, 4]` 를 처리한다.



포크 조인 풀(ForkJoinPool)

16:02:19.425 [ForkJoinPool-1-worker-2] [분할] [1, 2, 3, 4] -> LEFT[1, 2], RIGHT[3, 4]

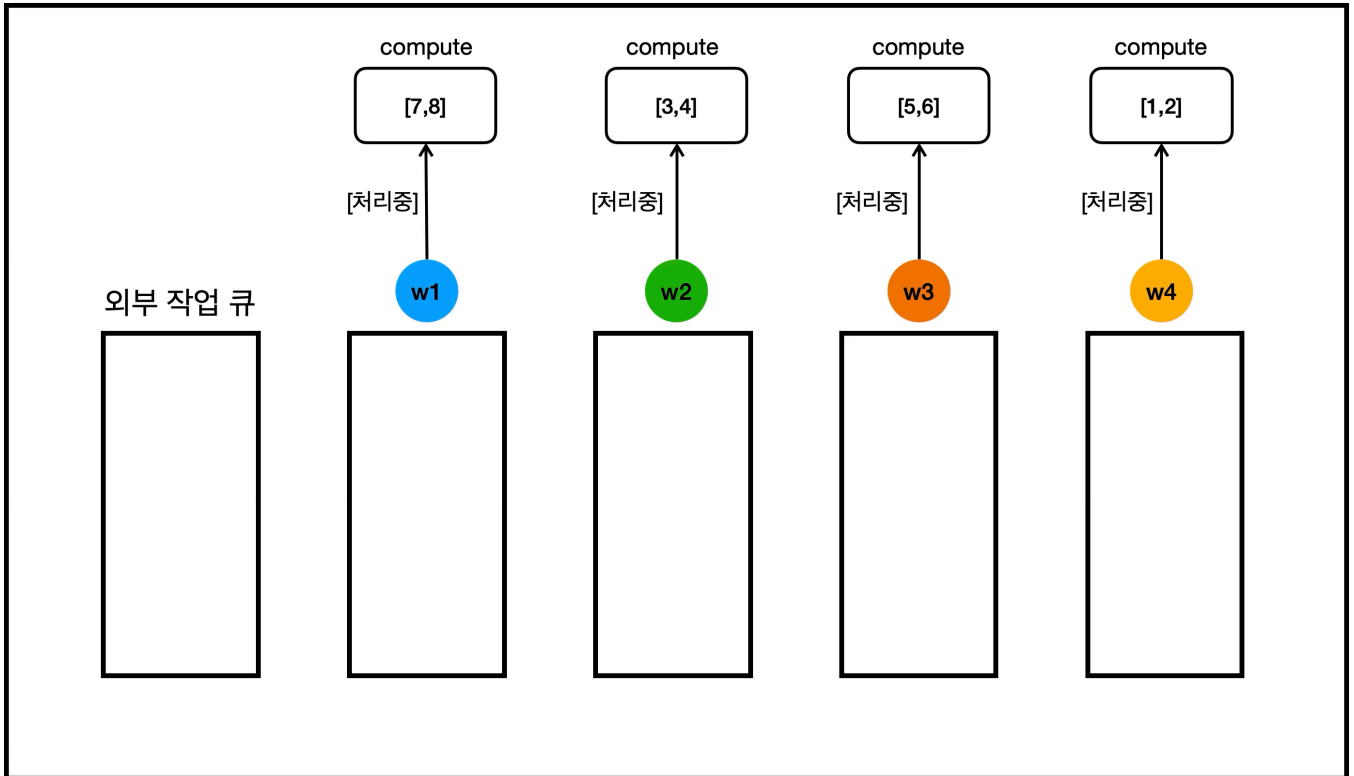
- w2 는 작업의 크기가 크다고 평가하고 작업을 둘로 분할한다.
- [1, 2] 의 작업은 fork를 호출해서 자신의 작업 큐에 넣어둔다.
- [3, 4] 의 작업은 compute를 호출해서 스스로 처리한다. (재귀 호출)



포크 조인 풀(ForkJoinPool)

작업 큐에 남아있는 작업들을 w3, w4 스레드가 훔쳐가서 실행한다.

- w3: w1의 작업 큐 [5,6]을 훔쳐서 처리
- w4: [w2]의 작업 큐 [1,2]를 훔쳐서 처리



포크 조인 풀(ForkJoinPool)

```
16:02:19.425 [ForkJoinPool-1-worker-1] [처리 시작] [7, 8]
16:02:19.425 [ForkJoinPool-1-worker-2] [처리 시작] [3, 4]
16:02:19.425 [ForkJoinPool-1-worker-3] [처리 시작] [5, 6]
16:02:19.425 [ForkJoinPool-1-worker-4] [처리 시작] [1, 2]
```

- 결과적으로 4개의 작업이 4개의 스레드에 분할되어 동시에 수행된다.

```
16:02:21.456 [      main] pool:
java.util.concurrent.ForkJoinPool@57baeedf[Terminated, parallelism = 10, size
= 0, active = 0, running = 0, steals = 4, tasks = 0, submissions = 0]
```

- 이번 작업에서 총 4번의 작업 훔치기가 있었다.
- 마지막에 출력한 ForkJoinPool의 로그를 확인해보면 `steals = 4` 항목을 확인할 수 있다.

작업 훔치기 알고리즘

이 예제에서는 작업량이 균등하게 분배되었지만, 실제 상황에서 작업량이 불균형할 경우 작업 훔치기 알고리즘이 동작하여 유휴 스레드가 다른 바쁜 스레드의 작업을 가져와 처리함으로써 전체 효율성을 높일 수 있다.

정리

임계값을 낮춤으로써 작업이 더 잘게 분할되고, 그 결과 더 많은 스레드가 병렬로 작업을 처리할 수 있게 되었다. 이는

Fork/Join 프레임워크의 핵심 개념인 **분할 정복(Divide and Conquer)** 전략을 명확하게 보여준다. 적절한 임계값 설정은 병렬 처리의 효율성에 큰 영향을 미치므로, 작업의 특성과 시스템 환경에 맞게 조정하는 것이 중요하다.

Fork/Join 적절한 작업 크기 선택

참고: 이 부분은 크게 중요하지 않으므로 참고로만 알아두자.

너무 작은 단위로 작업을 분할하면 스레드 생성과 관리에 드는 오버헤드가 커질 수 있으며, 너무 큰 단위로 분할하면 병렬 처리의 이점을 충분히 활용하지 못할 수 있다.

이 예제에서는 스레드 풀의 스레드가 10개로 충분히 남기 때문에 1개 단위로 더 잘게 쪼개는 것이 더 나은 결과를 보여준다.

이렇게 하면 8개의 작업을 8개의 스레드가 동시에 실행할 수 있다. 따라서 1초만에 작업을 완료할 수 있다.

하지만 예를 들어 1 ~ 1000까지 처리해야 하는 작업이라면 어떨까? 1개 단위로 너무 잘게 쪼개면 1000개의 작업으로 너무 잘게 분할된다. 스레드가 10개이므로 한 스레드당 100개의 작업을 처리해야 한다. 이 경우 스레드가 작업을 찾고 관리하는 부분도 늘어나고, 분할과 결과를 합하는 과정의 오버헤드도 너무 크다. 1000개로 쪼개고, 쪼갠 1000개를 합쳐야 한다.

예) 1 ~ 1000까지 처리해야 하는 작업, 스레드는 10개

- 1개 단위로 쪼개는 경우: 1000개의 분할과 결합이 필요. 한 스레드당 100개의 작업 처리
- 10개 단위로 쪼개는 경우: 100개의 분할과 결합이 필요. 한 스레드당 10개의 작업 처리
- 100개 단위로 쪼개는 경우: 10개의 분할과 결합이 필요. 한 스레드당 1개의 작업 처리
- 500개 단위로 쪼개는 경우: 2개의 분할과 결합이 필요. 스레드 최대 2개 사용 가능

작업시간이 완전히 동일하게 처리된다고 가정하면 이상적으로는 한 스레드당 1개의 작업을 처리하는 것이 좋다. 왜냐하면 스레드를 100% 사용하면서 분할과 결합의 오버헤드도 최소화 할 수 있기 때문이다.

하지만 작업 시간이 다른 경우를 고려한다면 한 스레드당 1개의 작업 보다는 더 잘게 쪼개어 두는 것이 좋다. 왜냐하면 `ForkJoinPool`은 스레드의 작업이 끝나면 다른 스레드가 처리하지 못하고 대기하는 작업을 훑어서 처리하는 기능을 제공하기 때문이다.

따라서 쉬는 스레드 없이 최대한 많은 스레드를 활용할 수 있다.

그리고 실질적으로는 작업 시간이 완전히 균등하지 않은 경우가 많다. 작업별로 처리 시간이 다르고, 시스템 환경에 따라 스레드 성능도 달라질 수 있다. 이런 상황에서 최적의 임계값 선택을 위해 고려해야 할 요소들은 다음과 같다.

- **작업의 복잡성:** 작업이 단순하면 분할 오버헤드가 더 크게 작용한다. 작업이 복잡할수록 더 작은 단위로 나누는 것이 유리할 수 있다. 예를 들어 $1 + 2 + 3 + 4$ 의 아주 단순한 연산을 $1 + 2$, $3 + 4$ 로 분할하게 되면 분할

하고 합하는 비용이 더 든다.

- **작업의 균일성:** 작업 처리 시간이 불균일할수록 작업 훔치기(work stealing)가 효과적으로 작동하도록 적절히 작은 크기로 분할하는 것이 중요하다.
- **시스템 하드웨어:** 코어 수, 캐시 크기, 메모리 대역폭 등 하드웨어 특성에 따라 최적의 작업 크기가 달라진다.
- **스레드 전환 비용:** 너무 작은 작업은 스레드 관리 오버헤드가 증가할 수 있다.

적절한 작업의 크기에 대한 정답은 없지만, CPU 바운드 작업이라고 가정할 때, CPU 코어수에 맞추어 스레드를 생성하고, 작업 수는 스레드 수에 4 ~ 10배 정도로 생성하자. 물론 작업의 성격에 따라 다르다. 그리고 성능 테스트를 통해 적절한 값으로 조절하면 된다.

Fork/Join 프레임워크3 - 공용 풀

Fork/Join 공용 풀(Common Pool) 설명

자바 8에서는 공용 풀(Common Pool)이라는 개념이 도입되었는데, 이는 Fork/Join 작업을 위한 자바가 제공하는 기본 스레드 풀이다.

```
// 자바 8 이상에서는 공용 풀(common pool) 사용 가능
ForkJoinPool commonPool = ForkJoinPool.commonPool();
```

Fork/Join 공용 풀의 특징

- **시스템 전체에서 공유:** 애플리케이션 내에서 단일 인스턴스로 공유되어 사용된다.
- **자동 생성:** 별도로 생성하지 않아도 `ForkJoinPool.commonPool()` 을 통해 접근할 수 있다.
- **편리한 사용:** 별도의 풀을 만들지 않고도 `RecursiveTask / RecursiveAction` 을 사용할 때 기본적으로 이 공용 풀이 사용된다.
- **병렬 스트림 활용:** 자바 8의 병렬 스트림은 내부적으로 이 공용 풀을 사용한다. (뒤에서 설명한다.)
- **자원 효율성:** 여러 곳에서 별도의 풀을 생성하는 대신 공용 풀을 사용함으로써 시스템 자원을 효율적으로 관리할 수 있다.
- **병렬 수준 자동 설정:** 기본적으로 시스템의 가용 프로세서 수에서 1을 뺀 값으로 병렬 수준(parallelism)이 설정된다. 예를 들어 CPU 코어가 14개라면 13개의 스레드가 사용된다. (자세한 이유는 뒤에서 설명)

Fork/Join 공용 풀은 쉽게 이야기해서, 개발자가 편리하게 Fork/Join 풀을 사용할 수 있도록 자바가 기본으로 제공하는 Fork/Join 풀의 단일 인스턴스이다.

Fork/Join 공용 풀을 어떻게 사용하는지 코드로 알아보자.

```
package parallel.forkjoin;

import java.util.List;
import java.util.concurrent.ExecutionException;
import java.util.concurrent.ForkJoinPool;
import java.util.stream.IntStream;

import static util.MyLogger.log;

public class ForkJoinMain2 {

    public static void main(String[] args) throws ExecutionException,
        InterruptedException {
        int processorCount = Runtime.getRuntime().availableProcessors();
        ForkJoinPool commonPool = ForkJoinPool.commonPool();
        log("processorCount = " + processorCount + ", commonPool = " +
            commonPool.getParallelism());

        List<Integer> data = IntStream.rangeClosed(1, 8)
            .boxed()
            .toList();

        log("[생성] " + data);
        SumTask task = new SumTask(data);
        Integer result = task.invoke(); // 공용 풀 사용
        log("최종 결과: " + result);
    }
}
```

- 이 예제 코드에서는 이전 예제와 달리 명시적으로 `ForkJoinPool` 인스턴스를 생성하지 않고, 대신 공용 풀을 사용한다.
- 참고로 `ForkJoinPool.commonPool()` 코드는 단순히 공용 풀 내부의 상태를 확인하기 위해 호출했다. 해당 코드가 없어도 공용 풀을 사용한다.

공용 풀을 통한 실행

이전에 사용했던 예제에서는 다음과 같이 `ForkJoinPool` 을 생성한 다음에 `pool.invoke(task)` 를 통해 풀에 직접 작업을 요청했다.

```
ForkJoinPool pool = new ForkJoinPool(10);
SumTask task = new SumTask(data);
int result = pool.invoke(task);
```

이번 예제를 보면 풀에 작업을 요청하는 것이 아니라 `task.invoke()` 를 통해 작업(`RecursiveTask`)에 있는 `invoke()` 를 직접 호출했다. 따라서 코드만 보면 풀을 전혀 사용하지 않는 것 처럼 보인다.

```
SumTask task = new SumTask(data);
Integer result = task.invoke(); // 공용 풀 사용
```

- 여기서 사용한 `invoke()` 메서드는 현재 스레드(여기서는 메인 스레드)에서 작업을 시작하지만, `fork()` 로 작업 분할 후에는 공용 풀의 워커 스레드들이 분할된 작업을 처리한다.
 - 메인 스레드가 스레드 풀이 아닌 `RecursiveTask`의 `invoke()` 를 직접 호출하면 메인 스레드가 작업의 `compute()` 를 호출하게 된다. 이때 내부에서 `fork()` 를 호출하면 공용 풀의 워커 스레드로 작업이 분할 된다.
- 메인 스레드는 최종 결과가 나올 때 까지 대기(블로킹)해야 한다. 따라서 그냥 대기하는 것 보다는 작업을 도와주는 편이 더 효율적이다.
 - `invoke()`: 호출 스레드가 작업을 도우면서 대기(블로킹)한다. 작업의 결과를 반환 받는다.
 - `fork()`: 작업을 비동기로 호출하려면 `invoke()` 대신에 `fork()` 를 호출하면 된다. `Future` (`ForkJoinTask`)를 반환 받는다.

실행 결과

```
10:33:55.237 processorCount = 14, commonPool = 13

10:33:55.240 [      main] [생성] [1, 2, 3, 4, 5, 6, 7, 8]
10:33:55.249 [      main] [분할] [1, 2, 3, 4, 5, 6, 7, 8] ->
                                     LEFT[1, 2, 3, 4], RIGHT[5, 6,
7, 8]
10:33:55.250 [      main] [분할] [5, 6, 7, 8] -> LEFT[5, 6], RIGHT[7, 8]
10:33:55.250 [worker-1] [분할] [1, 2, 3, 4] -> LEFT[1, 2], RIGHT[3, 4]
10:33:55.250 [      main] [처리 시작] [7, 8]
10:33:55.250 [worker-2] [처리 시작] [5, 6]
10:33:55.250 [worker-1] [처리 시작] [3, 4]
10:33:55.250 [worker-3] [처리 시작] [1, 2]
10:33:55.254 [worker-1] calculate 3 -> 30
10:33:55.254 [worker-2] calculate 5 -> 50
10:33:55.254 [worker-3] calculate 1 -> 10
```

```

10:33:55.254 [      main] calculate 7 -> 70
10:33:56.254 [ worker-1] calculate 4 -> 40
10:33:56.254 [ worker-3] calculate 2 -> 20
10:33:56.254 [ worker-2] calculate 6 -> 60
10:33:56.254 [      main] calculate 8 -> 80
10:33:57.258 [ worker-2] [처리 완료] [5, 6] -> sum: 110
10:33:57.258 [      main] [처리 완료] [7, 8] -> sum: 150
10:33:57.259 [      main] LEFT[110] + RIGHT[150] -> sum: 260
10:33:57.260 [ worker-1] [처리 완료] [3, 4] -> sum: 70
10:33:57.260 [ worker-3] [처리 완료] [1, 2] -> sum: 30
10:33:57.260 [ worker-1] LEFT[30] + RIGHT[70] -> sum: 100
10:33:57.260 [      main] LEFT[100] + RIGHT[260] -> sum: 360
10:33:57.260 [      main] 최종 결과: 360

```

- 로그의 양을 줄이기 위해 `ForkJoinPool.commonPool-` 은 생략했다.

스레드 수

```
processorCount = 14, commonPool = 13
```

- `processorCount = 14` 현재 CPU의 코어 수 이다.
- `parallelism = 13`: 동시에 처리할 수 있는 작업 수준(스레드 수와 관련)
 - 현재 CPU 코어가 14개(`processorCount = 14`)이다. 따라서 공용 풀은 CPU - 1의 수 만큼 스레드를 생성한다.
 - 여기서는 최대 13개의 스레드를 생성해서 사용한다. (이유는 뒤에서 설명)

작업 실행 과정

- 메인 스레드와 워커 스레드들이 함께 작업을 처리한다.
- 워커 스레드 이름이 `worker-1`, `worker-2`, `worker-3` 으로 표시된다(로깅 시 `ForkJoinPool.commonPool-` 접두사 생략)
- 메인 스레드도 작업 처리에 참여하는 것을 볼 수 있다(`[main]` 표시).

정리

- 공용 풀은 JVM이 종료될 때까지 계속 유지되므로, 별도로 풀을 종료(`shutdown()`)하지 않아도 된다.
- 이렇게 공용 풀(`ForkJoinPool.commonPool`)을 활용하면, 별도로 풀을 생성/관리하는 코드를 작성하지 않아도 간편하게 병렬 처리를 구현할 수 있다.

공용 풀 vs 커스텀 풀

이전 예제에서는 다음과 같이 커스텀 Fork/Join 풀을 생성했다.

```
ForkJoinPool pool = new ForkJoinPool();
Integer result = pool.invoke(task);
```

반면 이번 예제에서는 공용 풀을 사용했다.

```
Integer result = task.invoke();
```

차이점

1. **자원 관리**: 커스텀 풀은 명시적으로 생성하고 관리해야 하지만, 공용 풀은 시스템에서 자동으로 관리된다.
2. **재사용성**: 공용 풀은 여러 곳에서 공유할 수 있어 자원을 효율적으로 사용할 수 있다.
3. **설정 제어**: 커스텀 풀은 병렬 수준(스레드의 숫자), 스레드 팩토리 등을 세부적으로 제어할 수 있지만, 공용 풀은 기본 설정을 사용한다.
4. **라이프사이클**: 커스텀 풀은 명시적으로 종료해야 하지만, 공용 풀은 JVM이 관리한다. 따라서 종료하지 않아도 된다.

설정 변경

공용 풀 설정은 시스템 속성으로 변경할 수 있지만 권장하지는 않는다.

```
-Djava.util.concurrent.ForkJoinPool.common.parallelism=3
```

```
System.setProperty("java.util.concurrent.ForkJoinPool.common.parallelism",
    "3");
```

공용 풀이 CPU - 1만큼 스레드를 생성하는 이유

기본적으로 자바의 Fork/Join 공용 풀은 시스템의 가용 CPU 코어 수

(`Runtime.getRuntime().availableProcessors()`)에서 1을 뺀 값을 병렬 수준(parallelism)으로 사용한다. 예를 들어 CPU가 14코어라면 공용 풀은 최대 13개의 워커 스레드를 생성한다. 그 이유는 다음과 같다.

메인 스레드의 참여

Fork/Join 작업은 공용 풀의 워커 스레드뿐만 아니라 메인 스레드도 연산에 참여할 수 있다. 메인 스레드가 단순히 대

기하지 않고 직접 작업을 도와주기 때문에, 공용 풀에서 스레드를 14개까지 만들 필요 없이 13개의 워커 스레드 + 1개의 메인 스레드로 충분히 CPU 코어를 활용할 수 있다.

다른 프로세스와의 자원 경쟁 고려

애플리케이션이 실행되는 환경에서는 OS나 다른 애플리케이션, 혹은 GC(가비지 컬렉션) 같은 내부 작업들도 CPU를 사용해야 한다. 모든 코어를 최대치로 점유하도록 설정하면 다른 중요한 작업이 지연되거나, 컨텍스트 스위칭 비용(context switching)이 증가할 수 있다. 따라서 하나의 코어를 여유분으로 남겨 두어 전체 시스템 성능을 보다 안정적으로 유지하려는 목적이 있다.

효율적인 자원 활용

일반적으로는 CPU 코어 수와 동일하게 스레드를 만들더라도 성능상 큰 문제는 없지만, 공용 풀에서 CPU 코어 수 - 1을 기본값으로 설정함으로써, 특정 상황(다른 작업 스레드나 OS 레벨 작업)에서도 병목을 일으키지 않는 선에서 효율적으로 CPU를 활용할 수 있다.

자바 병렬 스트림

드디어 자바의 병렬 스트림(`parallel()`)을 사용해보자.

병렬 스트림은 Fork/Join 공용 풀을 사용해서 병렬 연산을 수행한다.

예제4 - 자바 병렬 스트림

```
package parallel;

import java.util.concurrent.ForkJoinPool;
import java.util.stream.IntStream;

import static util.MyLogger.log;

public class ParallelMain4 {

    public static void main(String[] args) {
        int processorCount = Runtime.getRuntime().availableProcessors();
        ForkJoinPool commonPool = ForkJoinPool.commonPool();
        log("processorCount = " + processorCount + ", commonPool = " +
        commonPool.getParallelism());
    }
}
```

```

    long startTime = System.currentTimeMillis();

    int sum = IntStream.rangeClosed(1, 8)
        .parallel() // 추가
        .map(HeavyJob::heavyTask)
        .reduce(0, (a, b) -> a + b);

    long endTime = System.currentTimeMillis();
    log("time: " + (endTime - startTime) + "ms, sum: " + sum);
}
}

```

- 예제 1에서 `parallel()` 하나만 추가했다.
- 실행 결과를 보면 여러 스레드가 병렬로 해당 업무를 처리한 것을 알 수 있다.

실행 결과

```

14:56:32.773 [      main] processorCount = 14, commonPool = 13

14:56:32.777 [      main] calculate 6 -> 60
14:56:32.777 [ForkJoinPool.commonPool-worker-2] calculate 2 -> 20
14:56:32.777 [ForkJoinPool.commonPool-worker-3] calculate 4 -> 40
14:56:32.777 [ForkJoinPool.commonPool-worker-1] calculate 3 -> 30
14:56:32.777 [ForkJoinPool.commonPool-worker-5] calculate 7 -> 70
14:56:32.778 [ForkJoinPool.commonPool-worker-6] calculate 1 -> 10
14:56:32.778 [ForkJoinPool.commonPool-worker-4] calculate 8 -> 80
14:56:32.778 [ForkJoinPool.commonPool-worker-7] calculate 5 -> 50
14:56:33.783 [      main] time: 1006ms, sum: 360

```

- 로그를 보면 `ForkJoinPool.commonPool-worker-N` 스레드들이 동시에 일을 처리하고 있다.
- 예제1에서 8초 이상 걸렸던 작업이, 이 예제에서는 모두 병렬로 실행되어 시간이 약 1초로 크게 줄어든다.
 - 만약 CPU 코어가 4개라면 공용 풀에는 3개의 스레드가 생성된다. 따라서 시간이 더 걸릴 수 있다.
- **직접 스레드를 만들 필요 없이** 스트림에 `parallel()` 메서드만 호출하면, 스트림이 자동으로 병렬 처리된다.

어떻게 복잡한 멀티스레드 코드 없이, `parallel()` 단 한 줄만 선언했는데, 해당 작업들이 병렬로 처리될 수 있을까? 바로 앞서 설명한 공용 `ForkJoinPool`을 사용하기 때문이다.

스트림에서 `parallel()`를 선언하면 스트림은 공용 `ForkJoinPool`을 사용하고, 내부적으로 병렬 처리 가능한 스레드 숫자와 작업의 크기 등을 확인하면서, `Splitter`를 통해 데이터를 자동으로 분할한다. 분할 방식은 데이터 소스의 특성에 따라 최적화되어 있다. 그리고 공용 풀을 통해 작업을 적절한 수준으로 분할(Fork), 처리(Execute)하고, 그 결과를 모은다(Join)

이때 요청 스레드(여기서는 메인 스레드)도 어차피 결과가 나올 때 까지 대기해야 하기 때문에, 작업에 참여해서 작업을 도운다.

개발자가 스트림을 병렬로 처리하고 싶다고 `parallel()` 로 선언만 하면, 실제 어떻게 할지는 자바 스트림이 내부적으로 알아서 처리하는 것이다!

코드를 보면 복잡한 멀티스레드 코드 하나 없이 `parallel()` 단 한 줄만 추가했다. 이것이 바로 람다 스트림을 활용한 선언적 프로그래밍 방식의 큰 장점이다.

병렬 스트림 사용시 주의점1

스트림에 `parallel()` 을 추가하면 병렬 스트림이 된다. 병렬 스트림은 Fork/Join 공용 풀을 사용한다. Fork/Join 공용 풀은 CPU 바운드 작업(계산 집약적인 작업)을 위해 설계되었다. 따라서 스레드가 주로 대기해야 하는 I/O 바운드 작업에는 적합하지 않다.

- I/O 바운드 작업은 주로 네트워크 호출을 통한 대기가 발생한다. 예) 외부 API 호출, 데이터베이스 조회

주의사항 - Fork/Join 프레임워크는 CPU 바운드 작업에만 사용해라!

Fork/Join 프레임워크는 주로 CPU 바운드 작업(계산 집약적인 작업)을 처리하기 위해 설계되었다. 이러한 작업은 CPU 사용률이 높고 I/O 대기 시간이 적다. CPU 바운드 작업의 경우, 물리적인 CPU 코어와 비슷한 수의 스레드를 사용하는 것이 최적의 성능을 발휘할 수 있다. 스레드 수가 코어 수보다 많아지면 컨텍스트 스위칭 비용이 증가하고, 스레드 간 경쟁으로 인해 오히려 성능이 저하될 수 있기 때문이다.

따라서, I/O 작업처럼 블로킹 대기 시간이 긴 작업을 `ForkJoinPool` 에서 처리하면 다음과 같은 문제가 발생한다.

1. 스레드 블로킹에 따른 CPU 낭비

- `ForkJoinPool` 은 CPU 코어 수에 맞춰 제한된 개수의 스레드를 사용한다. (특히 공용 풀)
- I/O 작업으로 스레드가 블로킹되면 CPU가 놀게 되어, 전체 병렬 처리 효율이 크게 떨어진다.

2. 컨텍스트 스위칭 오버헤드 증가

- I/O 작업 때문에 스레드를 늘리면, 실제 연산보다 대기 시간이 길어지는 상황이 발생할 수 있다.
- 스레드가 많아질수록 컨텍스트 스위칭(context switching) 비용도 증가하여 오히려 성능이 떨어질 수 있다.

3. 작업 훔치기 기법 무력화

- `ForkJoinPool` 이 제공하는 작업 훔치기 알고리즘은, CPU 바운드 작업에서 빠르게 작업 단위를 계속 처리하도록 설계되었다. (작업을 훔쳐서 쉬는 스레드 없이 계속 작업)

- I/O 대기 시간이 많은 작업은 스레드가 I/O로 인해 대기하고 있는 경우가 많아, 작업 훔치기가 빛을 발휘하기 어렵고, 결과적으로 병렬 처리의 장점을 살리기 어렵다.

4. 분할-정복(작업 분할) 이점 감소

- Fork/Join 방식을 통해 작업을 잘게 나누어도, I/O 병목이 발생하면 CPU 병렬화 이점이 크게 줄어든다.
- 오히려 분할된 작업들이 각기 I/O 대기를 반복하면서, `fork()`, `join()`에 따른 오버헤드만 증가할 수 있다.

정리

공용 풀(Common Pool)은 Fork/Join 프레임워크의 편리한 기능으로, 별도의 풀 생성 없이도 효율적인 병렬 처리를 가능하게 한다. 하지만 블로킹 작업이나 특수한 설정이 필요한 경우에는 커스텀 풀을 고려해야 한다.

- CPU 바운드 작업이라면 `ForkJoinPool`을 통해 병렬 계산을 극대화할 수 있지만, I/O 바운드 작업은 별도의 전용 스레드 풀을 사용하는 편이 더 적합하다.
- 예) `Executors.newFixedThreadPool()` 등등

병렬 스트림 - 예제5

예제를 통해 병렬 스트림 사용 시 주의점을 알아보자. 특히 여러 요청이 동시에 들어올 때 공용 풀에서 어떤 문제가 발생할 수 있는지 알아보자.

이 예제는 다음과 같은 시나리오를 시뮬레이션 한다.

1. 여러 사용자가 동시에 서버를 호출하는 상황
2. 각 요청은 병렬 스트림을 사용하여 몇 가지 무거운 작업을 처리
3. 모든 요청이 동일한 공용 풀(`ForkJoinPool.commonPool`)을 공유

```
package parallel;

import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.stream.IntStream;

import static util.MyLogger.log;

public class ParallelMain5 {

    public static void main(String[] args) throws InterruptedException {
        // 병렬 수준 3으로 제한

        System.setProperty("java.util.concurrent.ForkJoinPool.common.parallelism",
            "3");
```

```

// 요청 풀 추가
ExecutorService requestPool = Executors.newFixedThreadPool(100);

int nThreads = 3; // 1, 2, 3, 10, 20
for (int i = 1; i <= nThreads; i++) {
    String requestName = "request" + i;
    requestPool.submit(() -> logic(requestName));
    Thread.sleep(100); // 스레드 순서를 확인하기 위해 약간 대기
}
requestPool.close();
}

private static void logic(String requestName) {
    log "[" + requestName + "] START");
    long startTime = System.currentTimeMillis();

    int sum = IntStream.rangeClosed(1, 4)
        .parallel() // 주석 처리가 더 빨리 처리될 수 있음
        .map(i -> HeavyJob.heavyTask(i, requestName))
        .reduce(0, (a, b) -> a + b);

    long endTime = System.currentTimeMillis();
    log "[" + requestName + "] time: " + (endTime - startTime) + "ms, sum: " + sum);
}
}

```

- 여러분의 CPU 코어가 4개라고 가정하자. 시스템 속성을 사용해 공용 풀의 병렬 수준을 3으로 제한했다.
 - 공용 풀의 스레드 수가 3개가 된다.
- 예제를 단순화 하기 위해 1 ~ 4 범위의 작업을 처리한다. `IntStream.rangeClosed(1, 4)`
- `requestPool`은 여러 사용자 요청을 시뮬레이션하기 위한 스레드 풀이다.
- 각 요청은 `logic()` 메서드 안에서 `parallel()` 스트림을 사용하여 작업을 처리한다. 이때 공용 풀이 사용된다.

실행 결과

```

15:15:44.307 [pool-1-thread-1] [request1] START
15:15:44.316 [ForkJoinPool.commonPool-worker-3] [request1] 1 -> 10
15:15:44.316 [pool-1-thread-1] [request1] 3 -> 30
15:15:44.316 [ForkJoinPool.commonPool-worker-1] [request1] 2 -> 20
15:15:44.316 [ForkJoinPool.commonPool-worker-2] [request1] 4 -> 40

```

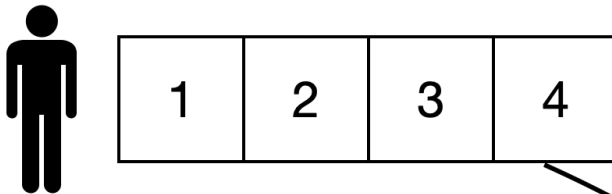
```

15:15:44.395 [pool-1-thread-2] [request2] START
15:15:44.395 [pool-1-thread-2] [request2] 3 -> 30
15:15:44.496 [pool-1-thread-3] [request3] START
15:15:44.496 [pool-1-thread-3] [request3] 3 -> 30
15:15:45.320 [ForkJoinPool.commonPool-worker-2] [request2] 2 -> 20
15:15:45.320 [ForkJoinPool.commonPool-worker-3] [request2] 4 -> 40
15:15:45.321 [ForkJoinPool.commonPool-worker-1] [request2] 1 -> 10
15:15:45.327 [pool-1-thread-1] [request1] time: 1012ms, sum: 100
15:15:45.501 [pool-1-thread-3] [request3] 4 -> 40
15:15:46.324 [ForkJoinPool.commonPool-worker-1] [request3] 2 -> 20
15:15:46.326 [ForkJoinPool.commonPool-worker-3] [request3] 1 -> 10
15:15:46.326 [pool-1-thread-2] [request2] time: 1931ms, sum: 100
15:15:47.332 [pool-1-thread-3] [request3] time: 2836ms, sum: 100

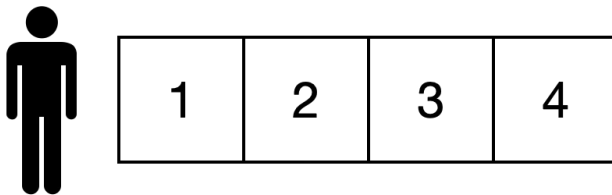
```

- 실행 결과는 환경에 따라 달라질 수 있음

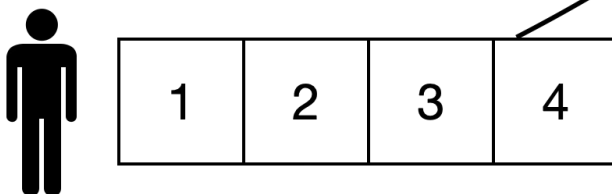
요청 그림1



요청 스레드1



요청 스레드2

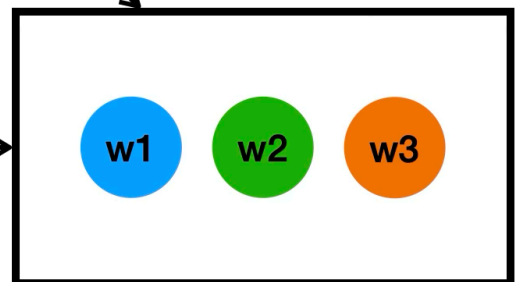


요청 스레드3

작업 요청

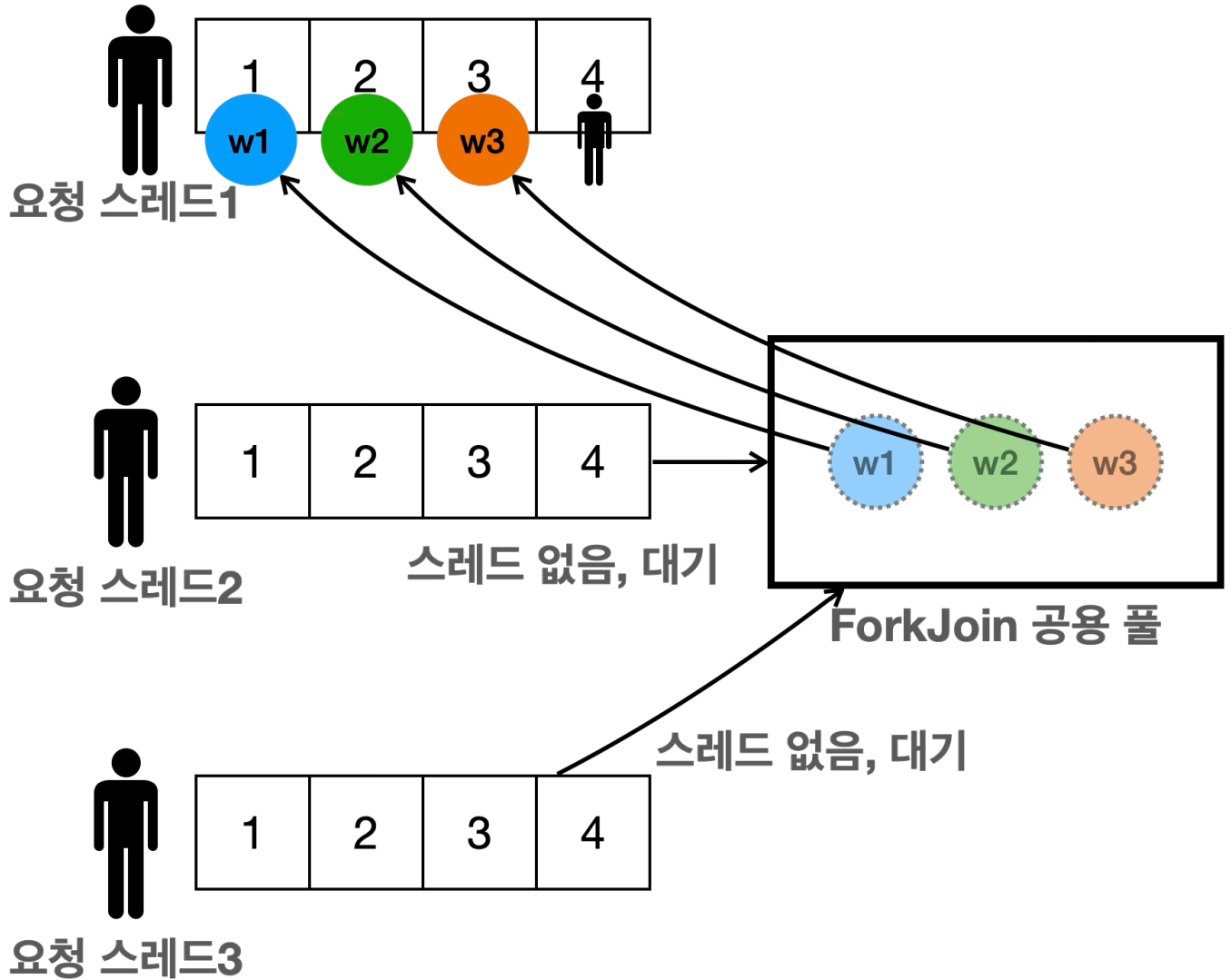
작업 요청

작업 요청



ForkJoin 공용 풀

요청 그림2



- 공용 풀의 제한된 병렬성

- 공용 풀은 병렬 수준(parallelism)이 3으로 설정되어 있어, 최대 3개의 작업만 동시에 처리할 수 있다. 여기에 요청 스레드도 자신의 작업에 참여하므로 각 작업당 총 4개의 스레드만 사용된다.
- 따라서 총 12개의 요청(각각 4개의 작업)을 처리하는데 필요한 스레드 자원이 부족하다.

- 처리 시간의 불균형

- request1: 1012ms (약 1초)
- request2: 1931ms (약 2초)
- request3: 2836ms (약 3초)
- 첫 번째 요청은 거의 모든 공용 풀 워커를 사용할 수 있었지만, 이후 요청들은 제한된 공용 풀 자원을 두고 경쟁해야 한다. 따라서 완료 시간이 점점 느려진다.

- 스레드 작업 분배

- 일부 작업은 요청 스레드(pool-1-thread-N)에서 직접 처리되고, 일부는 공용 풀(ForkJoinPool.commonPool-worker-N)에서 처리된다.

- 요청 스레드가 작업을 도와주지만, 공용 풀의 스레드가 매우 부족하기 때문에 한계가 있다.

요청이 증가할수록 이 문제는 더 심각해진다. `nThreads`의 숫자를 늘려서 동시 요청을 늘리면, 응답 시간이 확연하게 늘어나는 것을 확인할 수 있다.

핵심 문제점

- **공용 풀 병목 현상:** 모든 병렬 스트림이 동일한 공용 풀을 공유하므로, 요청이 많아질수록 병목 현상이 발생한다.
- **자원 경쟁:** 여러 요청이 제한된 스레드 풀을 두고 경쟁하면서 요청의 성능이 저하된다.
- **예측 불가능한 성능:** 같은 작업이라도 동시에 실행되는 다른 작업의 수에 따라 처리 시간이 크게 달라진다.

특히 실무 환경에서는 주로 여러 요청을 동시에 처리하는 애플리케이션 서버를 사용하게 된다. 이때 수 많은 요청이 공용 풀을 사용하는 것은 매우 위험할 수 있다. 따라서 병렬 스트림을 남용하면 전체 시스템 성능이 저하될 수 있다.

`nThreads`를 너무 늘리는 것 보다, 차라리 `parallel()`을 제거하는 것이 작업이 더 빨리 처리 될 수 있다.

`nThreads`를 20으로 설정한 다음 실행해보면 매우 느린 응답을 확인할 수 있다. 이때 차라리 `parallel()`을 제거하면 더 빠른 응답을 확인할 수 있다.

참고로 이번 예제에서 사용한 `heavyTask()`는 1초간 스레드가 대기하는 작업이다. 따라서 I/O 바운드 작업에 가깝다. 이런 종류의 작업은 Fork/Join 공용 풀 보다는 별도의 풀을 사용하는 것이 좋다.

주의! 실무에서 공용 풀은 절대! I/O 바운드 작업을 하면 안된다!

실무에서 공용 풀에 I/O 바운드 작업을 해서 장애가 나는 경우가 있다.

CPU 코어가 4개라면 공용 풀은 3개의 스레드만 사용한다. 그리고 공용 풀을 애플리케이션 전체에서 사용된다. 공용 풀에 있는 스레드 3개가 I/O 바운드 작업으로 대기하는 순간, 애플리케이션에서 공용 풀을 사용하는 모든 요청이 다 밀리게 된다.

예를 들어 공용 풀을 통해 외부 API를 호출하거나 데이터베이스를 호출하고 기다리는 경우가 있다. 만약 외부 API나 데이터베이스의 응답이 늦게 온다면 공용 풀의 3개의 스레드가 모두 I/O 응답을 대기하게 된다. 그리고 나머지 모든 요청이 공용 풀의 스레드를 기다리며 다 밀리게 되는 무시무시한 일이 발생한다.

공용 풀은 반드시 CPU 바운드(계산 집약적인) 작업에만 사용해야 한다!

병렬 스트림은 처음부터 Fork/Join 공용 풀을 사용해서 CPU 바운드 작업에 맞도록 설계되어 있다.

따라서 이런 부분을 잘 모르고 실무에서 병렬 스트림에 I/O 대기 작업을 하는 것은 아주 위험한 일이다.

특히 병렬 스트림의 경우 단순히 `parallel()` 한 줄만 추가하면 병렬 처리가 되기 때문에, 어떤 스레드가 사용되는지도 제대로 이해하지 못하고 사용하는 경우가 있다. 병렬 스트림은 반드시 CPU 바운드 작업에만 사용하자!

그렇다면 여러 작업을 병렬로 처리해야 하는데, I/O 바운드 작업이 많을 때는 어떻게 하면 좋을까? 이때는 스레드를 직접 사용하거나, `ExecutorService` 등을 통해 별도의 스레드 풀을 사용해야 한다.

병렬 스트림 사용시 주의점2

별도의 풀 사용

별도의 전용 스레드 풀을 사용해서 앞선 예제의 문제를 해결해보자.

병렬 스트림 - 예제6

```
package parallel;

import java.util.concurrent.*;

import static util.MyLogger.log;

public class ParallelMain6 {

    public static void main(String[] args) throws InterruptedException {
        ExecutorService requestPool = Executors.newFixedThreadPool(100);

        // logic 처리 전용 스레드 풀 추가
        ExecutorService logicPool = Executors.newFixedThreadPool(400);

        int nThreads = 3; // 1, 2, 3, 10, 20
        for (int i = 1; i <= nThreads; i++) {
            String requestName = "request" + i;
            requestPool.submit(() -> logic(requestName, logicPool));
            Thread.sleep(100); // 스레드 순서를 확인하기 위해 약간 대기
        }

        requestPool.close();
        logicPool.close();
    }

    private static void logic(String requestName, ExecutorService es) {
        log("[ " + requestName + " ] START");
        long startTime = System.currentTimeMillis();
    }
}
```

```

        // 1부터 4까지의 작업을 각각 스레드 풀에 제출
        Future<Integer> f1 = es.submit(() -> HeavyJob.heavyTask(1,
requestName));
        Future<Integer> f2 = es.submit(() -> HeavyJob.heavyTask(2,
requestName));
        Future<Integer> f3 = es.submit(() -> HeavyJob.heavyTask(3,
requestName));
        Future<Integer> f4 = es.submit(() -> HeavyJob.heavyTask(4,
requestName));

        // Future 결과 취합
        int sum;
        try {
            Integer v1 = f1.get();
            Integer v2 = f2.get();
            Integer v3 = f3.get();
            Integer v4 = f4.get();
            sum = v1 + v2 + v3 + v4;
        } catch (Exception e) {
            throw new RuntimeException(e);
        }

        long endTime = System.currentTimeMillis();
        log "[" + requestName + "] time: " + (endTime - startTime) + "ms, sum:
" + sum);
    }
}

```

변경 사항

- 전용 로직 풀 추가
 - `ExecutorService logicPool = Executors.newFixedThreadPool(400)`
 - 최대 400개의 스레드를 가진 별도의 풀을 생성해서 병렬 작업 처리에 사용한다.
- 병렬 스트림 대신 커스텀 스레드 풀 사용
 - 병렬 스트림(`parallel()`)을 사용하지 않고, 직접 전용 스레드 풀에 작업을 제출한다.
 - 각 작업이 별도의 `Future`로 관리된다.
- 결과 취합 방식
 - `Future.get()`을 사용하여 각 작업의 결과를 기다렸다가 취합한다.

실행 결과


```

15:42:46.880 [pool-1-thread-1] [request1] START
15:42:46.886 [pool-2-thread-2] [request1] 2 -> 20
15:42:46.886 [pool-2-thread-3] [request1] 3 -> 30
15:42:46.886 [pool-2-thread-1] [request1] 1 -> 10
15:42:46.886 [pool-2-thread-4] [request1] 4 -> 40
15:42:46.964 [pool-1-thread-2] [request2] START
15:42:46.964 [pool-2-thread-5] [request2] 1 -> 10
15:42:46.964 [pool-2-thread-6] [request2] 2 -> 20
15:42:46.964 [pool-2-thread-7] [request2] 3 -> 30
15:42:46.965 [pool-2-thread-8] [request2] 4 -> 40
15:42:47.069 [pool-1-thread-3] [request3] START
15:42:47.069 [pool-2-thread-9] [request3] 1 -> 10
15:42:47.069 [pool-2-thread-10] [request3] 2 -> 20
15:42:47.069 [pool-2-thread-11] [request3] 3 -> 30
15:42:47.070 [pool-2-thread-12] [request3] 4 -> 40
15:42:47.892 [pool-1-thread-1] [request1] time: 1007ms, sum: 100
15:42:47.965 [pool-1-thread-2] [request2] time: 1001ms, sum: 100
15:42:48.074 [pool-1-thread-3] [request3] time: 1005ms, sum: 100

```

- **일관된 처리 시간**
 - 모든 요청이 약 1초 내외로 처리되었다.
 - 이전 예제에서 관찰된 요청별 처리 시간이 지연되는 문제가 해결되었다.
- **독립적인 스레드 할당**
 - 각 요청의 작업들이 전용 풀(pool-2-thread-N)에서 처리된다.
 - Fork/Join 공용 풀 스레드(ForkJoinPool.commonPool-worker-N)가 사용되지 않았다.
- **확장성 향상**
 - 400개의 스레드를 가진 풀을 사용함으로써, 동시에 여러 요청을 효율적으로 처리한다.
 - 공용 풀 병목 현상이 발생하지 않았다.

이 예제는 작업 유형에 적합한 전용 스레드 풀을 사용하는 것의 이점을 보여준다. 특히 I/O 바운드 작업이나 많은 동시 요청을 처리하는 서버 환경에서는 이러한 접근 방식이 더 효과적이다.

병렬 스트림 - 예제7

앞선 예제는 쉽게 설명하기 위해 코드를 풀어서 사용했다. 간결하게 개선해보자.

```

package parallel;

import java.util.List;
import java.util.concurrent.ExecutorService;

```

```

import java.util.concurrent.Executors;
import java.util.concurrent.ForkJoinPool;
import java.util.concurrent.Future;
import java.util.stream.IntStream;

import static util.MyLogger.log;

public class ParallelMain7 {

    public static void main(String[] args) throws InterruptedException {
        ExecutorService requestPool = Executors.newFixedThreadPool(100);

        // logic 처리 전용 스레드 풀 추가
        ExecutorService logicPool = Executors.newFixedThreadPool(400);

        int nThreads = 3; // 1, 2, 3, 10, 20
        for (int i = 1; i <= nThreads; i++) {
            String requestName = "request" + i;
            // 각 요청을 스레드 풀에 제출
            requestPool.submit(() -> logic(requestName, logicPool));
            Thread.sleep(100); // 스레드 순서를 확인하기 위한 대기
        }

        // 모든 작업 제출 후 스레드 풀 종료
        requestPool.close();
        logicPool.close();
    }

    private static void logic(String requestName, ExecutorService es) {
        log("[ " + requestName + " ] START");
        long startTime = System.currentTimeMillis();

        // 1부터 4까지의 작업을 각각 스레드 풀에 제출
        List<Future<Integer>> futures = IntStream.rangeClosed(1, 4)
            .mapToObj(i -> es.submit(() -> HeavyJob.heavyTask(i,
requestName)))
            .toList();

        // Future 결과 취합
        int sum = futures.stream()
            .mapToInt(f -> {
                try {
                    return f.get();

```

```

        } catch (Exception e) {
            throw new RuntimeException(e);
        }
    }).sum();

    long endTime = System.currentTimeMillis();
    log "[" + requestName + "] time: " + (endTime - startTime) + "ms, sum: " + sum);
}
}

```

- **스트림 API 활용**

- 개별 Future 변수(f1, f2, f3, f4) 대신 IntStream과 mapToObj를 사용하여 작업 제출을 간결화

- **코드 간결화**

- 이전 예제에 비해 로직이 더 명확하고 간결해짐
- 작업 수가 변경되어도 코드 수정이 최소화

실행 결과

```

15:43:07.844 [pool-1-thread-1] [request1] START
15:43:07.852 [pool-2-thread-4] [request1] 4 -> 40
15:43:07.852 [pool-2-thread-3] [request1] 3 -> 30
15:43:07.852 [pool-2-thread-2] [request1] 2 -> 20
15:43:07.852 [pool-2-thread-1] [request1] 1 -> 10
15:43:07.928 [pool-1-thread-2] [request2] START
15:43:07.929 [pool-2-thread-5] [request2] 1 -> 10
15:43:07.929 [pool-2-thread-6] [request2] 2 -> 20
15:43:07.929 [pool-2-thread-7] [request2] 3 -> 30
15:43:07.929 [pool-2-thread-8] [request2] 4 -> 40
15:43:08.032 [pool-1-thread-3] [request3] START
15:43:08.032 [pool-2-thread-9] [request3] 1 -> 10
15:43:08.032 [pool-2-thread-10] [request3] 2 -> 20
15:43:08.032 [pool-2-thread-11] [request3] 3 -> 30
15:43:08.032 [pool-2-thread-12] [request3] 4 -> 40
15:43:08.855 [pool-1-thread-1] [request1] time: 1007ms, sum: 100
15:43:08.933 [pool-1-thread-2] [request2] time: 1004ms, sum: 100
15:43:09.037 [pool-1-thread-3] [request3] time: 1005ms, sum: 100

```

실행 결과는 기존과 같다.

정리

여기까지 단일 스트림부터 시작해서 직접 스레드 생성, 스레드 풀, Fork/Join 프레임워크, 그리고 자바 병렬 스트림까지 차근차근 살펴보았다. 병렬 처리를 위한 전반적인 흐름과 주의해야 할 점은 다음과 같다.

1. 단일 스트림 vs 멀티스레드

- **단일 스트림**: 코드가 간단하지만, 한 번에 하나의 스레드만 실행되어 시간이 오래 걸린다.
- **멀티스레드**: 여러 스레드를 직접 생성해 병렬로 작업을 처리할 수 있으나, 스레드 생성, 관리, 예외 처리 등이 복잡해진다.

2. 스레드 풀(ExecutorService)

- 스레드를 직접 생성, 제어하는 대신, 자바가 제공하는 스레드 풀을 활용해 멀티스레드를 더 쉽게 사용할 수 있다.
- `submit(Callable)` 과 `Future` 를 통해 작업을 비동기로 처리하고, 결과를 손쉽게 받아올 수 있다.

3. Fork/Join 패턴과 Fork/Join 프레임워크

- 큰 작업을 잘게 **분할(Fork)** 한 뒤 여러 스레드가 **병렬로 처리**하고, 최종 결과를 **합치기(Join)** 하는 전형적인 병렬 처리 패턴이다.
- 자바의 Fork/Join 프레임워크(`ForkJoinPool`, `RecursiveTask`, `RecursiveAction`)는 이러한 패턴을 편리하게 지원한다.
- **작업 훔치기(Work-Stealing)** 알고리즘을 통해 각 스레드가 할당받은 작업이 없으면, 다른 스레드의 큐에 있는 작업을 훔쳐서 효율적으로 분산 처리한다.
- **CPU 바운드** 작업(계산 집약적)일 때 최적의 효과를 낸다.

4. 자바 병렬 스트림

- 람다 스트림에서 `parallel()` 한 줄만 추가해도 내부적으로 Fork/Join 공용 풀을 사용하여 자동으로 병렬 처리한다.
- 개발자는 복잡한 멀티스레드 코드를 짤 필요 없이 선언적으로 병렬 연산을 수행할 수 있다.
- **단점**: 공용 풀을 공유하므로, I/O 대기 작업이나 동시 요청이 많아지는 상황에서 병목 현상이 발생할 수 있다.

5. 병렬 스트림 사용 시 주의사항

- **CPU 바운드(계산 집약적) 작업**에만 사용하는 것이 권장된다.
- **I/O 바운드 작업**(DB 조회, 외부 API 호출 등)은 오랜 대기 시간이 발생하므로, 제한된 스레드만 쓰는

Fork/Join 공용 풀과 궁합이 좋지 않다.

- 서버 환경에서 여러 요청이 동시에 병렬 스트림을 사용하면 공용 풀이 빠르게 포화되어 전체 성능이 저하될 수 있다(특히 I/O 바운드 작업을 병렬 스트림으로 사용하면 더 큰 문제).

6. 별도의 풀 사용

- I/O 바운드 작업처럼 대기가 긴 경우에는 **전용 스레드 풀(ExecutorService)**을 만들어 사용하는 것을 권장한다.
- 스레드 풀의 크기, 스레드 생성 정책, 큐 타입 등을 상황에 맞게 튜닝할 수 있어 확장성과 안정성이 높아진다.

결론

- 자바 병렬 스트림은 CPU 바운드 작업을 빠르고 쉽게 병렬 처리하기에는 좋지만, 공용 풀을 이용한다는 점에서 I/O 바운드나 동시 요청이 많은 시스템에는 주의가 필요하다.
- 병렬 스트림만으로 모든 병렬 처리를 해결하려 하기보다는, **작업 특성**(CPU 바운드 vs I/O 바운드)과 **시스템 환경**(서버 동시 요청, 리소스 상황)을 고려하여 **Fork/Join** 프레임워크 또는 **별도의 스레드 풀**을 적절히 조합해 사용하는 것이 중요하다.

CompletableFuture와 주의 사항

특히 실무에서 자주 하는 실수가 병렬 스트림을 I/O 대기 작업에 사용하거나, 또는 `CompletableFuture`를 사용할 때 발생한다.

- 병렬 스트림은 처음부터 Fork/Join 공용 풀과 함께 사용하도록 설계되었다. 따라서 반드시 CPU 바운드 작업에 사용해야 한다.
 - I/O 대기 작업에 사용하면 Fork/Join 공용 풀의 스레드 병목으로 모든 요청이 밀리는 큰 장애가 발생하기 쉽다.
- 실무에서 복잡한 멀티스레드 코드를 작성할 때는 `CompletableFuture`가 도움이 된다.
- 우리는 멀티스레드, 스레드 풀, `Future`, 랴다, 스트림을 모두 학습했기 때문에 어렵지 않게 `CompletableFuture`를 이해할 수 있을 것이다. (`CompletableFuture`는 실무에서 필요할 때 찾아서 학습하면 충분하다.)
- `CompletableFuture`를 생성할 때는 별도의 스레드 풀을 반드시 지정해야 한다. 그렇지 않으면 **Fork/Join** 공용 풀이 대신 사용된다. 이 때문에 많은 장애가 발생한다. `CompletableFuture`를 사용할 때는 **반드시! 커스텀 풀을 지정해서 사용하자!**

CompletableFuture의 주의점

```
package parallel.forkjoin;
```

```

import java.util.concurrent.CompletableFuture;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;

import static util.MyLogger.*;

public class CompletableFutureMain {

    public static void main(String[] args) {
        CompletableFuture.runAsync(() -> log("Fork/Join")); // Fork/Join 공용 풀

        ExecutorService es = Executors.newFixedThreadPool(100);
        CompletableFuture.runAsync(() -> log("Custom Pool"), es); // 별도의 풀
        es.close();
    }
}

```

- 첫 번째 예제는 별도의 스레드 풀을 지정하지 않았다.
- 두 번째 예제는 별도의 스레드 풀(es)을 지정했다.

실행 결과

```

18:11:13.137 [pool-1-thread-1] Custom Pool
18:11:13.137 [ForkJoinPool.commonPool-worker-1] Fork/Join

```

- CompletableFuture에 스레드 풀을 지정하지 않으면 Fork/Join 공용 풀이 사용되는 것을 확인할 수 있다.